



Bahir Dar University

Bahir Dar Institution of Technology

Faculty of Computing

Requirements Analysis Document (RAD) and Requirements Design Document
for

Digital waste management system

Submitted to the faculty of computing in partial fulfillment of the requirements for the degree of
Bachelor of Science in **Software Engineering**

Group Members

Group members	ID Number
Wondesen Getaneh	BDU1404092
Wbamlak Grum	BDU1404373
Hibritu Diress	BDU1404467

Advisor : ALEBACHEW.M

[Year: 2025]

Bahir Dar University, Bahir Dar Institute of Technology

DECLARATION

- The Project is our own work and has not been presented for a degree in any other university. All sources of material used for the project have been duly acknowledged.

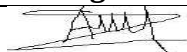
Name	Signature
Wondesen Getaneh	
Wbamlak Grum	
Hibritu Diress	

Faculty: Computing

Program: Software Engineering

Project Title: **Digital waste management System**

This is to certify that I have read this project and that in my supervision and the students' performance, it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

Name of Advisor	Signature
ALEBACHEW M.	

Examining Committee Members:

#	Name	Signature	Date
1	Examiner 1	_____	_____
2	Examiner 2	_____	_____

- It is approved that this project has been written in compliance with the formatting rules laid down by the faculty.

Roles and Responsibilities of the Group Members

- The table below shows the participation of each group member in the project tasks. Each tick (√) indicates active participation. This ensures that all members understand the project work by sharing experiences.

List of Tasks	Wondesen Getaneh	Wbamlak Grum	hbritu
Task 1: Requirement Gathering	√	√	√
Task 2: System Analysis & Design	√		
Task 3: Database Design			√
Task 4: UI Prototyping	√		
Task 5: Implementation	√	√	√
Task 6: Testing & Evaluation	√	√	√

Table I.I Roles and Responsibility of group members

Acknowledgement

We would like to express our sincere and deepest gratitude to all individuals who contributed, directly or indirectly, to the successful completion of this project. First and foremost, we extend our heartfelt appreciation to our supervisor for the continuous guidance, professional advice, constructive criticism, and unwavering support provided throughout the entire project development process. Their expertise and encouragement played a significant role in shaping the direction and quality of this work.

We are also grateful to the faculty members for providing essential academic resources, valuable insights, and constructive feedback at various stages of the project. Their contributions greatly enhanced our understanding and helped improve the overall quality and relevance of the system developed. In addition, we would like to thank our classmates and colleagues for their meaningful discussions, helpful suggestions, cooperation, and motivation, which made the project journey more productive and engaging.

Finally, we extend our sincere appreciation to our family and friends for their patience, moral support, and constant encouragement throughout the project period. Their understanding and support gave us the strength and motivation to overcome challenges and successfully complete this project. This work would not have been possible without the assistance, encouragement, and support of all those mentioned above.

List of Acronyms

- ACID - Atomic, Consistent, Isolated, Durable
- AES - Advanced Encryption Standard
- App - Application
- API - Application Program Interface
- AWS - Amazon Web Services
- CRC - Class-Responsibility-Collaboration
- DB - Database
- DBMS - Database Management System
- DoS - Denial-of-Service
- GPS - Global Positioning System
- HDD - Hard Disk Drive
- HSTS - HTTP Strict Transport Security
- IO - Input/Output
- JWT - JSON Web Tokens
- LTS - Long Term Support
- NFRs - Non-Functional Requirements
- OOAD - Object-Oriented Analysis and Design
- ORM - Object-Relational Mapping
- OS - Operating System
- QM - Queue Manager
- RAD - Requirements Analysis Document
- RAM - Random Access Memory
- RBAC - Role-Based Access Control
- RDD - Requirements Definition Document
- SSL/TLS - Secure Sockets Layer/Transport Layer Security
- SSD - Solid State Drive
- TLS - Transport Layer Security
- UI - User Interface
- UI/UX - User Interface/User Experience
- UML - Unified Modeling Language
- UPS - Uninterruptible Power Supply
- UX - User Experience
- UML - Unified Modeling Language
- UPS - Uninterruptible Power Supply
- UX - User Experience
- WCAG - Web Content Accessibility Guidelines

List of Figures

Figure 1:USE-CASE DIAGRAM	22
Figure 2:State Chart Diagram	25
Figure 3:Activity Diagram.....	26
Figure 4:Sequence Diagram.....	27
Figure 5:Class Diagram	33
Figure 6:Component Diagram	44
Figure 7:Deployment Diagram	46
Figure 8:E-R Diagram.....	49
Figure 9:Login Page.....	51
Figure 10:Citizen Dashboard	52
Figure 11:Association Leader Dashboard.....	53
Figure 12 :City Admin Dashboard.....	53
Figure 13:Sub-City Admin Dashboard	54
Figure 14:Sub-City kebeles page	54
Figure 15:Sub-City Admin kebeles list page	55
Figure 16:UT-01	70
Figure 17:U-02.....	71
Figure 18:UT-03	71
Figure 19:UT-04	72

List of Tables

Table 1:Detail Class with Attribute28

Table 2:CRC (Class-Responsibility-Collaborator)31

Table 3:Persistent Tables and Mappings.....49

Table 4: Unit Test Cases69

Table 5:Sample Integration Test CasesTable 6:System Test Case73

Table 7:System Test Case74

Table 8:Security Test Cases.....75

Table 9:Test Result76

Table of Contents

Contents

- Examining Committee Members:2
- List of Acronyms.....5
- 1. User Registration and Authentication:.....20
 - 2. Demand Reporting:.....20
 - 3. Collection Scheduling:.....20
 - 4. Route Optimization:.....20
 - 5. Monitoring and Tracking:.....20
 - 6. Notifications and Alerts:20
 - 7. Reporting and Analytics:20
 - 8. System Administration:.....20
- 1. Citizen Use Cases:21
 - 2. Waste Collector Use Cases:21
 - 3. Municipal Staff Use Cases:.....21
- 1. Role-Based Access:23
 - 2. Dynamic Scheduling Rule:.....23
 - 3. Reporting Rule:23
 - 4. Notification Rule:23
 - 5. Data Integrity Rule:23
 - 6. Operational Limitations Rule:23
- 1. Citizen Interface:23
 - 2. Waste Collector Interface:24
 - 3. Municipal Staff Interface:24
- 1. Citizen Reporting Workflow:.....24
 - 2. Waste Collector Workflow:.....24
 - 3. Municipal Staff Workflow:25
- 1. Performance:29
 - 2. Scalability:29

3. Reliability & Availability:	29
4. Security:	30
5. Usability:	30
6. Maintainability:	30
7. Portability:.....	30
Notes:.....	32
1. Submit Waste Request.....	33
○ State Changes:	33
2. Assign Collection	33
○ State Changes:	34
3. Update Collection Status	34
○ State Changes:	34
4. Deliver Waste to Recycling Center	34
○ State Changes:	34
5. Generate Report	34
○ State Changes:	34
1. Customer Interface	34
2. Staff Interface	35
3. Admin Interface	35
3. System Design Overview	35
3.1 Design Goals	36
1. Replace Fixed-Day Waste Collection with Demand-Based Scheduling	36
2. Improve Resource Utilization through Route, Vehicle, and Workforce Optimization	36
3. Support Real-Time Monitoring and Coordination among Stakeholders	36
4. Enable Data-Driven Planning and Decision-Making	37
5. Ensure Scalability, Security, Usability, and Maintainability.....	37
3.2 Architectural Design	37
3.2 Component Design	40
3.2.2 Frontend Component Design (React)	41
3.2.3 Backend Component Design (Node.js and Express).....	42
3.2.4 Database Component Design	42
3.2.5 Component Interaction Summary	43
3.1.7 Deployment Modeling.....	44

3.2.1 Design Class Model	46
Class Relationships and Multiplicity	48
3.2.2 Persistent Model	48
Persistent Tables and Mappings.....	49
Key Relationships (Foreign Keys).....	50
Persistent Model Characteristics.....	50
3.3 User Interface Design	50
3.3.1 Authentication and Landing Interface.....	51
3.3.2 Citizen(Customer) Interface (Request & Signal Portal)	52
3.3.3 Association Leader Interface (Operational View).....	52
3.3.5 Sub-City Admin Dashboard.....	53
3.6 Access Control and Security	55
User Roles and Access Levels	55
Security Mechanisms.....	56
Chapter Four: Implementation.....	56
4.1 Implementation Overview.....	56
The system supports multiple roles:	57
4.2 Backend Implementation	57
4.2.1 Admin	57
4.2.2 Authentication and authorization	58
Chapter Five: Testing and Evaluation	67
5.1 Overview of Testing and Evaluation	67
5.2 Testing Parameters (Quality Attributes).....	68
5.3 Unit Testing.....	69
Unit Test Cases	69
5.4 Integration Testing	72
Integration Test Cases	73
5.5 System Testing.....	73
System Test Cases.....	74
5.6 Security Testing	74
Security Test Cases	75
5.7 Evaluation Summary.....	75
Questionnaire and Interview Topics.....	77
1. Existing System: Municipal Waste Management	77
3. Existing System: Waste Collection Associations.....	77

4. Citizen Interaction and Service Requests	77
5. Priority Handling and Waste Collection Challenges	77
6. Communication and Notification Mechanisms	77
7. System Improvement Expectations.....	78

Abstract

Waste management is one of the major challenges faced by growing urban cities. In many cities, waste collection is handled through a manual, block-based system where waste collection associations are assigned fixed areas and operate on predetermined collection schedules. Although this approach ensures local responsibility and accountability, it lacks flexibility, real-time information, and data-driven decision-making. Fixed-day collection does not always reflect actual waste generation patterns, leading to inefficiencies such as delayed collection, unnecessary trips, and poor resource utilization.

This project proposes a **Digital Demand-Based Waste Management System** designed to enhance waste collection efficiency through centralized coordination and data-driven decision support. The proposed system enables dynamic collection planning based on actual demand, waste generation patterns, and available resources, rather than relying solely on fixed schedules. By integrating digital reporting, monitoring, and prioritization mechanisms, the system improves resource utilization, supports timely waste collection, and enhances overall city cleanliness.

The system is intended to support municipalities, waste collection associations, and citizens by providing better coordination, transparency, and decision-making capabilities. Unlike traditional manual systems, the proposed digital solution allows cities to scale waste management operations efficiently as urban growth and waste variability increase.

Chapter One – Introduction

1.1 BACKGROUND

Rapid urbanization and population growth have significantly increased the volume and complexity of solid waste generated in modern cities. Effective waste management has therefore become a critical public service that directly affects public health, environmental sustainability, and overall quality of life. Many cities, especially in our countries, rely on traditional manual waste collection systems that are based on fixed schedules, predefined service areas, and human experience.

In a typical manual waste management system, the bahir dar waste management system, the city is divided into administrative units, and waste collection associations are assigned specific areas of responsibility. Waste is collected on predetermined days according to fixed schedules, and operational decisions are largely based on prior experience rather than real-time data. While this approach provides clear accountability and local familiarity, it offers limited flexibility in responding to variations in waste generation across different areas and time periods.

As cities expand and waste generation patterns become increasingly dynamic due to population growth, commercial activities, seasonal events, and changes in consumption behavior, fixed schedule-based collection systems face growing challenges. These challenges include inefficient resource utilization, delayed response to waste accumulation, and difficulty in coordinating waste collection activities across wider urban areas. Furthermore, the absence of digital data limits the ability of municipalities to analyze waste trends, optimize resource allocation, and plan future waste management strategies effectively.

Advancements in information and communication technologies have enabled the development of digital waste management systems that support data-driven and demand-based collection approaches. Such systems allow municipalities to move beyond static collection schedules by incorporating real-time information, centralized coordination, and flexible decision-making. As a result, digital waste management solutions are increasingly being adopted to improve efficiency, scalability, and sustainability in urban waste collection operations.

1.2 STATEMENT OF THE PROBLEM

Urban waste collection in many cities is primarily managed through a manual, block-based system with fixed collection schedules. In this system, waste collection associations are assigned predefined service areas and operate according to predetermined collection days. While this approach provides clear responsibility and local familiarity, it presents several limitations as cities grow and waste generation patterns become more dynamic.

One of the major challenges of the existing manual system is its reliance on fixed schedules rather than actual waste demand. Waste generation varies significantly across different areas and time periods due to factors such as population density, commercial activities, seasonal events, and changing consumption behavior. However, fixed-day collection does not adapt to these variations, often resulting in delayed collection in high-demand areas and unnecessary collection trips in low-demand areas.

Additionally, the manual system lacks effective data-driven decision-making mechanisms. Information related to waste accumulation, collection frequency, resource usage, and service performance is not systematically captured or analyzed. As a result, municipalities depend heavily on experience and manual reporting, which limits their ability to optimize resource utilization, respond promptly to emerging waste issues, and plan for future waste management needs.

The absence of centralized coordination and real-time information further complicates efficient waste collection. Resources such as trucks and workforce are allocated based on static plans rather than actual demand conditions, making it difficult to balance workload and improve operational efficiency as urban areas expand.

Therefore, there is a need for a digital waste management solution that supports demand-based collection, enables data-driven decision-making, and enhances coordination without eliminating the accountability and responsibility structures of existing systems. Addressing these challenges is essential to improve efficiency, scalability, and sustainability in urban waste management.

1.3 OBJECTIVE OF THE PROJECT

1.3.1 General Objective

- The general objective of this project is to design and develop a **digital demand-based waste management system** that improves the efficiency of waste collection through data-driven decision-making, flexible resource utilization, and improved coordination among waste management stakeholders in urban cities.

1.3.2 Specific Objective

The specific objectives of the study are to:

1. Study and analyze the existing manual waste management and collection system.
2. Identify the limitations of fixed schedule-based and block-based waste collection approaches.
3. Design a digital system that supports demand-based waste collection planning.
4. Enable data-driven decision-making through centralized data collection and monitoring.
5. Improve utilization of waste collection resources such as trucks and workforce.
6. Enhance coordination between municipalities, waste collection associations, and citizens.
7. Support scalable waste management operations as cities grow and waste patterns become more dynamic.

1.2 METHODOLOGY OF THE PROJECT

- This study follows a **systematic and structured approach** to analyze the existing waste management system and design a digital demand-based solution.
- First, a detailed analysis of the existing manual waste management process is conducted through document review, stakeholder discussions, and observation of current collection practices. This helps identify operational limitations related to efficiency, scheduling, and resource utilization.
- Second, system requirements are gathered by identifying the needs of key stakeholders, including municipalities, waste collection associations, and citizens. Both functional and non-functional requirements are defined based on the identified problems.
- Third, a conceptual design of the digital waste management system is developed. The proposed system emphasizes demand-based collection, centralized data management, and decision-support mechanisms for planning and prioritization.
- Finally, appropriate tools and technologies are selected to design and prototype the system, followed by validation of the proposed solution against the identified requirements to ensure feasibility and effectiveness.

1.5 FEASIBILITY OF THE PROJECT

The feasibility of the proposed digital waste management system is evaluated from three perspectives: technical, economic, and operational.

Technical Feasibility:

The system can be developed using currently available technologies, including web and mobile platforms, database management systems, and cloud-based tools for data storage and analytics. The development team possesses the necessary skills in software development, system analysis, and database design to implement the solution successfully.

Economical Feasibility:

The proposed system is cost-effective compared to the current manual system. By optimizing collection routes, reducing unnecessary trips, and enabling data-driven decision-making, the system has the potential to reduce operational costs. Initial investment in system development and deployment is justified by the long-term savings and improved service efficiency.

Operational Feasibility:

The system aligns with the operational requirements of municipalities and waste collection associations. It is designed to be user-friendly for staff and citizens, ensures real-time monitoring of collection activities, and provides actionable insights for planning. The solution enhances service reliability and addresses the limitations of fixed-day collection schedules.

1.6 BENEFICIARIES OR SIGNIFICANCE OF THE PROJECT

- The proposed digital waste management system brings significant benefits to various stakeholders involved in waste collection and management.

Municipality and Local Authorities:

By providing real-time data on waste generation and collection, the system enables better planning, resource allocation, and monitoring of waste management activities. This leads to improved operational efficiency and accountability.

Waste collection associations:

The system allows waste collectors to optimize their routes and schedules based on actual demand rather than fixed collection days. This reduces fuel consumption, labor costs, and workload, while improving service coverage.

Citizens:

Residents benefit from timely and reliable waste collection services. The system also provides channels for reporting missed collections or overflow issues, promoting citizen participation and satisfaction.

Environment and Community:

By reducing unnecessary trips, optimizing collection frequency, and preventing waste accumulation, the system contributes to cleaner communities, reduced environmental pollution, and overall public health improvement.

Academic and Research institute:

The collected data can support studies and research on waste generation patterns, recycling potential, and sustainable urban planning.

1.7 LIMITATION OF THE PROJECT

- While the proposed digital waste management system offers significant improvements over the existing manual process, certain limitations are acknowledged:

Technical Limitation:

The system relies on internet connectivity and digital devices for real-time data collection and monitoring. In areas with limited network coverage or insufficient digital infrastructure, system performance may be affected.

Financial Limitation:

Although the system reduces operational costs in the long term, initial development, deployment, and maintenance require financial investment. Budget constraints may affect the speed of implementation and scalability.

Operational Limitation:

The effectiveness of the system depends on the active participation of waste collectors and citizens. Resistance to change from traditional practices or low user engagement may limit the system's potential benefits.

Data Limitation:

The accuracy of system outputs depends on the quality and consistency of the data entered. Incomplete or erroneous data input may reduce the reliability of insights and decision-making.

Scope Limitation:

The system focuses on municipal solid waste collection and may not fully address industrial or hazardous waste management requirements. Future expansions will be needed to cover additional waste categories.

1.8 SCOPE OF THE PROJECT

This study focuses on the analysis and design of a **digital demand-based waste management system** for urban environments. The scope of the project covers municipal solid waste generated from residential areas, commercial establishments, and public institutions that are part of the regular waste collection service.

The system is intended to support waste collection planning, demand-based scheduling, resource coordination, and monitoring at the city and sub-city levels. It includes interactions between municipalities, waste collection associations, and citizens.

The project does not include the management of hazardous, medical, or industrial waste, as these require specialized handling and are typically managed through separate systems. Physical implementation, procurement of hardware, and full-scale deployment are also outside the scope of this study.

The outcome of this study is a requirement analysis and system design that can be adapted and scaled for different cities.

1.9 ORGANIZATION OF THE PROJECT

This project report is organized into chapters and sections to provide a clear and structured presentation of the study, system design, and implementation.

Chapter One: Introduction

- Provides the background of the study, problem statement, objectives, and significance of the project.
- Discusses methodology, feasibility, limitations, and scope of the project.

Chapter Two: System Features

- Presents an analysis of the existing waste management system and identifies its limitations.
- Introduces the proposed digital waste management system and its advantages.
- Details the system requirements, including functional and non-functional requirements, use cases, business rules, and interface prototypes.

Chapter Three: System Design and Modeling

- Includes system design artifacts such as flowcharts, sequence diagrams, class diagrams,

activity diagrams, and state charts.

- Provides database design, logic models, and key abstractions for system development.

Chapter Four: Implementation

- Describes the development environment, tools, and technologies used.
- Explains system implementation steps, including installation, configuration, and operational procedures.

Chapter One: Testing and Result

- Presents system testing strategies, test cases, and results.
- Evaluates system performance and demonstrates how the system meets its objectives.

Chapter Six: Conclusion and Recommendations

- Summarizes findings, highlights benefits of the proposed system, and provides suggestions for future improvements.

Reference and Appendix

- Lists all cited literature, resources, and supporting documentation, including forms, questionnaires, and additional diagrams.

CHAPTER TWO: SYSTEM FEATURES

2.1 THE EXISTING SYSTEM

The current waste management system in most municipalities is largely manual and follows a fixed-day collection schedule. Waste collection is performed according to predetermined routes and days, regardless of the actual amount of waste generated in different neighborhoods.

Key Characteristics Of The Existing System:

- **Manual Scheduling:** Collection schedules are fixed and planned without real-time data on waste generation.
- **Inefficient Resource Utilization:** Waste trucks often travel routes with low waste volumes, resulting in unnecessary fuel consumption and labor costs.
- **Lack of Data-Driven Insights:** There is no centralized system to record, track, or analyze waste generation patterns. Decision-making is largely based on past experience rather than real-time information.
- **Limited Public Interaction:** Citizens have minimal channels to report missed collections or overflowing bins, leading to dissatisfaction and complaints.
- **Operational Challenges:** The system struggles to adapt to changes in population density, seasonal waste variations, or unexpected events like festivals or community clean-ups.

Problems Identified In The Existing System:

1. **Fixed-Day Collection Inefficiency:** Collecting waste on fixed days often results in over-collection in low-waste areas and under-collection in high-waste areas.
2. **High Operational Costs:** Fuel, labor, and maintenance costs are higher due to unnecessary trips and poor route optimization.

3. **Lack of Monitoring and Accountability:** Supervisors cannot monitor collection activities in real-time, making it difficult to track performance and ensure service quality.
 4. **Delayed Problem Resolution:** Overflowing bins or missed collections are addressed reactively, leading to environmental and public health concerns.
- ❖ The limitations of the existing system highlight the need for a **digital, demand-based waste management solution** that leverages data to improve efficiency, reduce costs, and enhance citizen satisfaction.

2.2 PROPOSED SYSTEM

The proposed system is a **digital, demand-based waste management system** designed to address the inefficiencies of the existing manual and fixed-day collection approach. The system leverages data-driven decision-making to improve waste collection efficiency, resource utilization, and service quality.

Unlike the existing system, the proposed solution enables **dynamic waste collection based on actual demand** rather than predefined schedules. It integrates municipalities, waste collection associations, and citizens into a centralized digital platform.

Key Features Of The Proposed System:

- **Demand-based collection:**
Waste collection schedules are generated based on real-time or reported waste levels, ensuring that areas with higher demand are prioritized while unnecessary trips are minimized.
- **Centralized Data Management:**
The system stores and manages data related to waste generation, collection frequency, routes, and service performance. This data supports analysis and informed decision-making.
- **Citizen Participation:**
Citizens can report issues such as overflowing bins, missed collections, or special waste disposal needs, improving communication and service responsiveness.
- **Real-Time Monitoring and Reporting:**
Municipal authorities can monitor collection activities, evaluate performance, and generate reports to support planning and accountability.

Benefits Of Proposed System

- Improved efficiency in waste collection operations
 - Reduced operational and transportation costs
 - Better utilization of manpower and vehicles
 - Enhanced cleanliness and environmental protection
 - Increased transparency and citizen satisfaction
- ❖ By transforming waste management from a manual and reactive process into a **digital and proactive system**, the proposed solution ensures sustainable, efficient, and reliable waste collection services.

2.3 REQUIREMENT ANALYSIS

2.3.1 FUNCTIONAL REQUIRMENTS

- The functional requirements define the key actions and operations that the digital waste management system must perform to meet stakeholder needs. These requirements ensure the system delivers its intended functionality effectively.

Key Functional Requirements:

- 1. User Registration and Authentication:**
 - Municipal staff, waste collectors, and citizens must be able to register and log in securely.
 - Role-based access ensures that each user type sees relevant features.
- 2. Demand Reporting:**
 - Citizens can report overflowing bins, missed collections, or special waste pickup requests via the system.
 - Reports are timestamped and linked to specific locations.
- 3. Collection Scheduling:**
 - The system generates dynamic collection schedules based on reported demand and historical data.
 - Routes are optimized for efficiency and minimal travel time.
- 4. Route Optimization:**
 - Waste collection vehicles are assigned to optimized routes automatically.
 - Routes can be updated in real-time based on new demand reports or operational changes.
- 5. Monitoring and Tracking:**
 - Municipal staff can monitor collection activities in real-time.
 - Waste collectors' progress on assigned routes is tracked and logged.
- 6. Notifications and Alerts:**
 - Automatic notifications are sent to citizens when their area is scheduled for waste collection.
 - Alerts are generated for missed pickups, delays, or operational issues.
- 7. Reporting and Analytics:**
 - The system generates performance reports, including collection frequency, route efficiency, and service coverage.
 - Data-driven insights help municipalities plan and allocate resources effectively.
- 8. System Administration:**
 - Administrators can manage user roles, update system parameters, and maintain overall system integrity.

2.3.2 System Use Case

- A **use case** defines how different users interact with the system to accomplish specific tasks. The proposed digital waste management system involves three main types of users: **Municipal Staff**, **Waste Collectors**, and **Citizens**.

Primary Use Case:

1. Citizen Use Cases:

- **Report Waste Issue:** Citizens report overflowing bins, missed collections, or special waste pickups.
- **Receive Notifications:** Citizens receive updates about collection schedules and completed services.
- **View Collection History:** Citizens can view past waste collection activities in their area.

2. Waste Collector Use Cases:

- **Receive Assigned Routes:** Collectors are assigned optimized routes based on real-time demand.
- **Update Collection Status:** Collectors mark waste collection tasks as completed in the system.
- **Report Issues:** Collectors can report operational issues, such as vehicle breakdowns or inaccessible areas.

3. Municipal Staff Use Cases:

- **Monitor Collection Activities:** Staff can track waste collection progress in real-time.
- **Generate Reports:** Staff can create reports on performance, service coverage, and operational efficiency.
- **Manage Users and System Settings:** Administrators manage user accounts, roles, and system configurations.

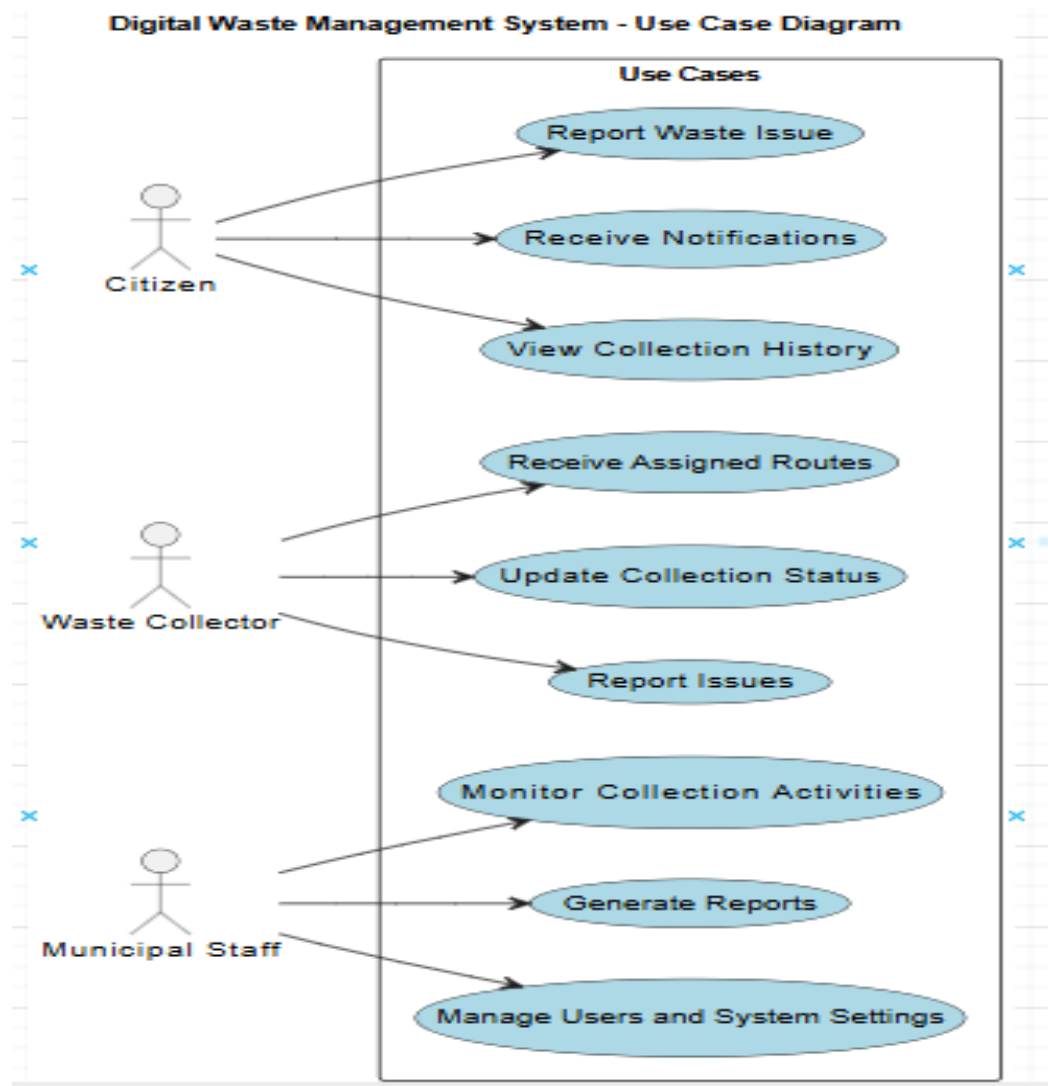


Figure 1:USE-CASE DIAGRAM

2.3.3 Business Rule Documentation

- Business rules define the constraints, policies, and guidelines that govern the operation of the digital waste management system. They ensure consistency, accuracy, and proper system behavior according to organizational and operational requirements.

Key Business Rule:

- 1. Role-Based Access:**
 - Each user type (Citizen, Waste Collector, Municipal Staff) can only access the features relevant to their role.
 - Administrators have full access to system configuration and user management.
- 2. Dynamic Scheduling Rule:**
 - Waste collection schedules are generated based on real-time reports and historical waste generation data.
 - Areas with higher reported waste levels are prioritized over low-demand areas.
- 3. Reporting Rule:**
 - Citizens must submit valid location information when reporting a waste issue.
 - Waste Collectors must update task status immediately after completing a collection.
- 4. Notification Rule:**
 - Citizens receive notifications only for their registered location.
 - Alerts for missed collections are sent automatically to municipal staff.
- 5. Data Integrity Rule:**
 - All reports, updates, and schedules must be timestamped and linked to user IDs for accountability.
 - Historical data cannot be deleted; it is archived for future reference.
- 6. Operational Limitations Rule:**
 - The system only manages municipal solid waste; industrial, medical, or hazardous waste is excluded.
 - Collection is performed only in areas within the municipality's operational boundaries.

Summary

These business rules ensure that the system operates predictably, supports efficient waste management, and enforces accountability among users while maintaining the integrity of the data and services provided.

2.3.4 User Interface Prototype

The User Interface (UI) prototype illustrates the visual design and user interactions for the digital waste management system. The prototype serves as a blueprint for developers and provides stakeholders with a clear understanding of how the system will operate.

Key Ui Components:

- 1. Citizen Interface:**
 - **Home Dashboard:** Shows upcoming collection schedules and recent reports.
 - **Report Waste Issue Form:** Allows users to submit location, waste type, and issue description.
 - **Notifications Panel:** Displays alerts about collection schedules and completed requests.
 - **History Page:** Lists previous submissions and status updates.

2. Waste Collector Interface:

- **Assigned Routes Dashboard:** Displays optimized collection routes for the day.
- **Task Status Update:** Allows marking each collection task as completed or reporting issues.
- **Alerts Panel:** Notifies collectors of changes to routes or urgent collection requests.

3. Municipal Staff Interface:

- **Monitoring Dashboard:** Shows real-time collection activity across different zones.
- **Reports Section:** Allows generating performance, route efficiency, and service coverage reports.
- **User Management:** Provides controls for adding, removing, or updating system users.
- **System Settings:** Configures system parameters such as notification rules and route optimization preferences.

Prototype Approach

- The prototype can be designed using wireframing tools like **Figma, Adobe XD, or Balsamiq**.
- Screens are designed to be **user-friendly, intuitive, and responsive**, ensuring that all users can perform their tasks efficiently.
- The UI prototype is **iterative**, allowing for adjustments based on stakeholder feedback before final implementation.

Summary

The UI prototype ensures that the system is **accessible, visually clear, and easy to navigate**, helping both citizens and municipal staff perform their tasks efficiently while supporting demand-based waste management operations.

2.3.5 State Chart Diagram

A **state chart diagram** illustrates the different states of the digital waste management system and how it transitions between these states based on user interactions or system events. This helps in understanding the system's dynamic behavior and workflow.

Key State and Transitions:

1. Citizen Reporting Workflow:

- **Idle:** Default state before any report is submitted.
- **Report Submitted:** When a citizen submits a waste issue.
- **Report Verified:** Municipal staff or the system verifies the report.
- **Collection Scheduled:** The system assigns the report to a collection route.
- **Completed:** The waste is collected, and the report is closed.

2. Waste Collector Workflow:

- **Idle:** Default state before receiving assigned routes.
- **Route Assigned:** When the system assigns a collection route.
- **Collection In Progress:** As the collector collects waste along the route.
- **Task Completed:** All assigned collection points on the route are completed.
- **Issue Reported:** If a problem occurs during collection, such as inaccessible areas or equipment failure.

3. Municipal Staff Workflow:

- **Monitoring Idle:** Default state before reviewing system dashboards.
- **Monitoring Active:** Real-time tracking of waste collection and reports.
- **Analysis:** Generating reports and analyzing data for planning.
- **Decision Making:** Adjusting schedules or assigning additional resources based on insights.

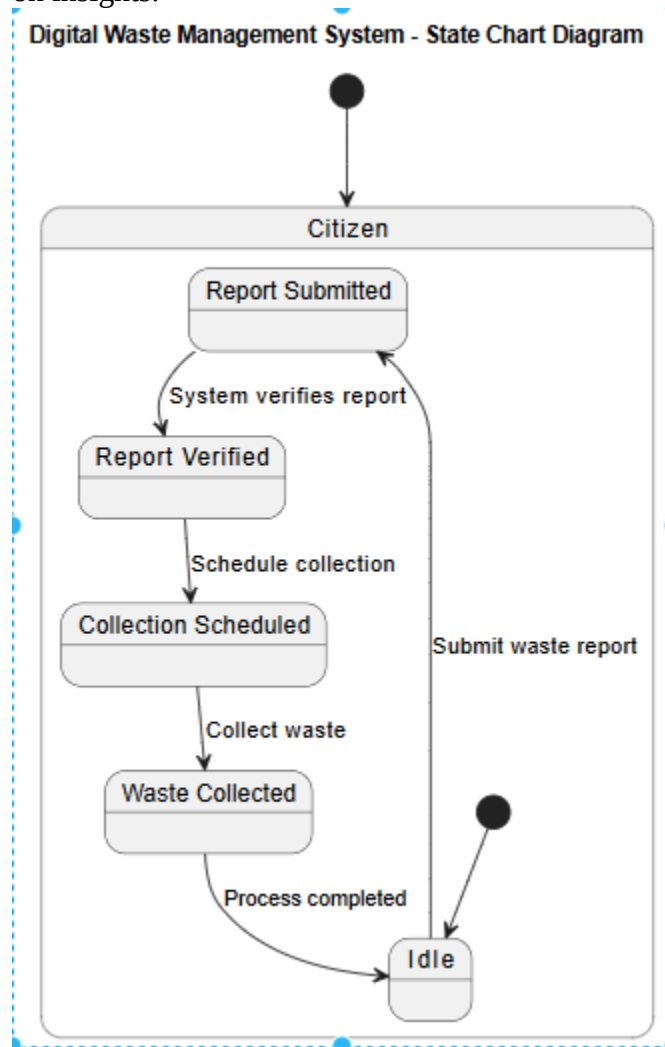


Figure 2:State Chart Diagram

Summary:

The state chart diagram ensures that the system's processes are **well-defined, traceable, and manageable**, allowing all users to understand how reports, tasks, and system activities progress from start to completion.

2.3.6 Activity Diagram

An activity diagram is used to illustrate the dynamic workflow of the digital waste management system by showing the sequence of activities performed by different actors and the system itself. It helps in understanding how tasks flow from one activity to another, from waste reporting to waste collection and completion.

In the proposed digital waste management system, the activity diagram represents how citizens initiate waste collection requests, how the system processes and verifies these requests, and how municipal authorities and waste collectors coordinate to complete the waste collection process efficiently.

The workflow begins when a citizen submits a waste collection request through the system. The system then validates the request and stores it in the centralized database. Based on priority, location, and resource availability, the system schedules a collection task and assigns it to an appropriate waste collection team. The waste collectors receive the assigned task, collect the waste from the specified location, and update the system upon completion. Finally, the system records the completion status and updates performance and monitoring data.

This activity-based workflow replaces the traditional fixed-day and manual waste collection process with a demand-driven, data-informed approach, ensuring efficient resource utilization and timely waste collection.

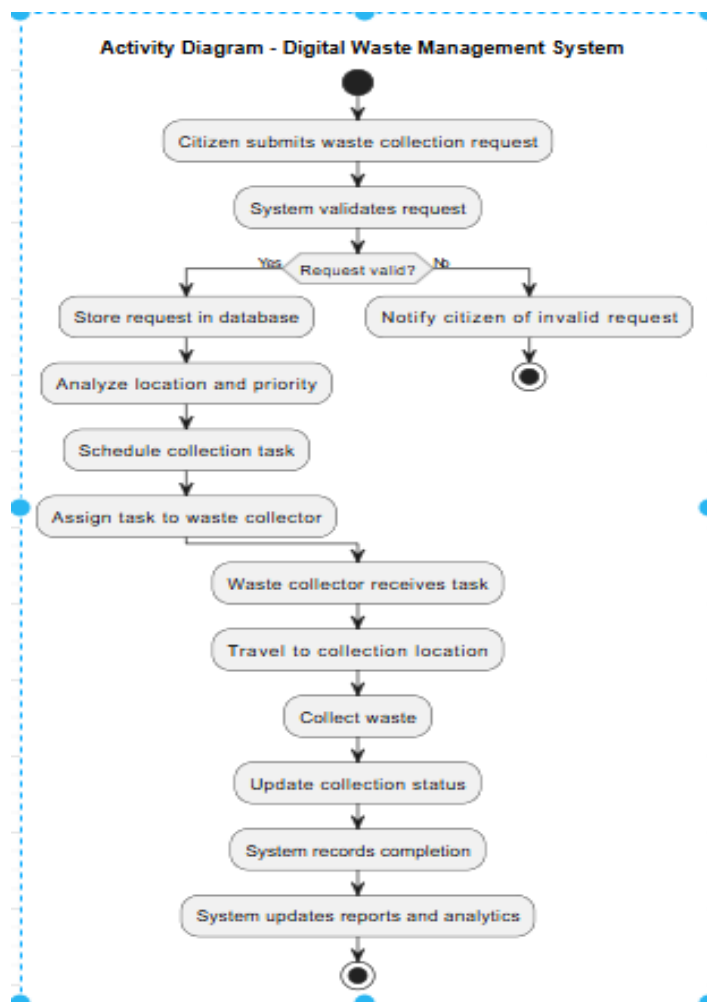


Figure 3:Activity Diagram

2.3.7 Sequence Diagram

A sequence diagram illustrates how different actors and system components interact with each other over time to accomplish a specific task. It focuses on the order of messages exchanged between actors and the system, making it easier to understand the system's behavior in real execution scenarios.

In the digital waste management system, the sequence diagram demonstrates how a waste collection request flows from the citizen to the system, how the system processes the request, assigns it to the municipal authority and waste collectors, and how feedback is returned after task completion.

The interaction starts when a citizen submits a waste collection request through the digital platform. The system validates and stores the request, then notifies municipal staff for review and scheduling. After scheduling, the system assigns the task to a waste collector. The collector performs the collection and updates the system, which finally notifies the citizen and updates system reports. This sequence replaces manual communication and fixed-day collection with automated, real-time coordination.

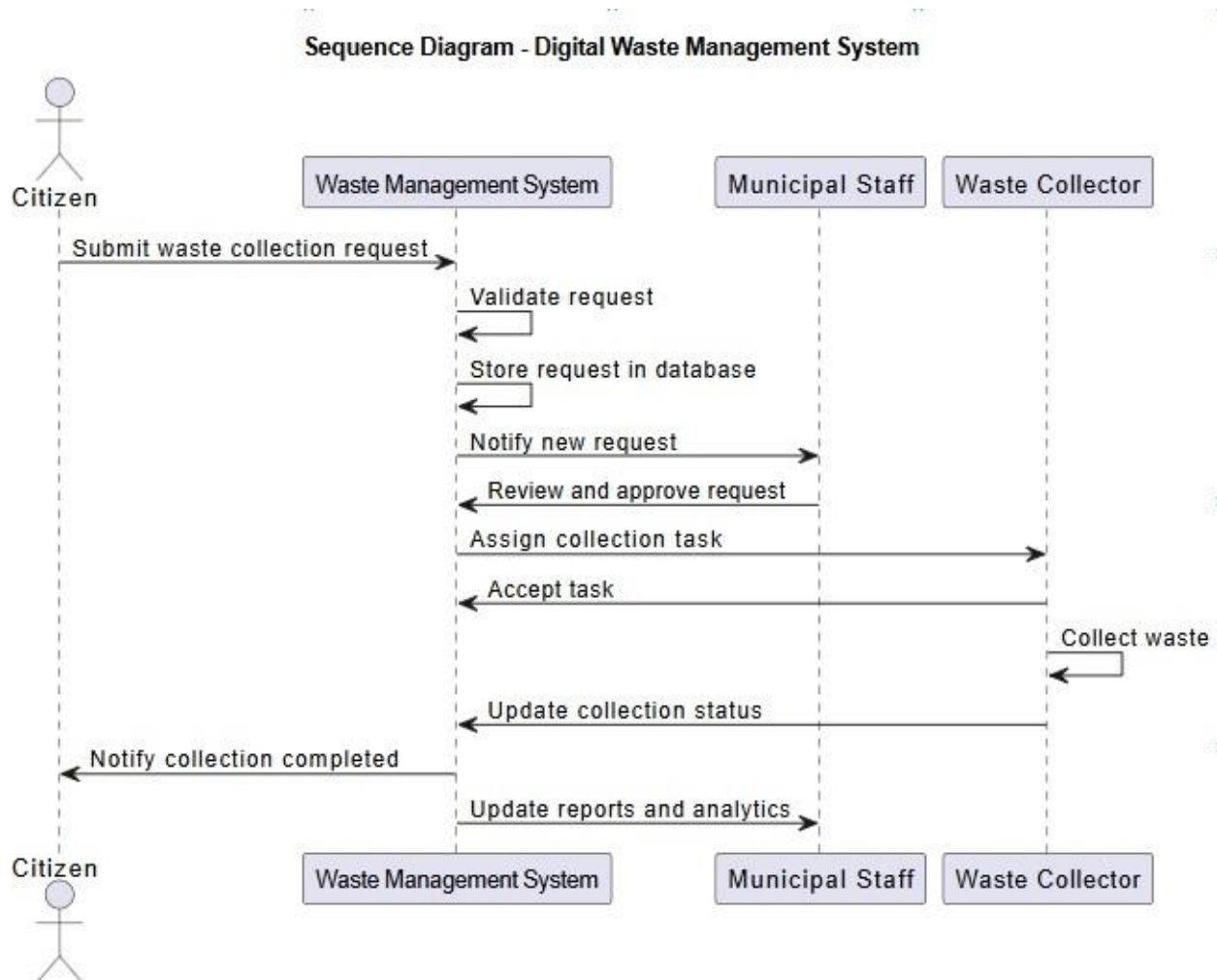


Figure 4:Sequence Diagram

2.3.8 Analysis Class Model

This model provides a **detailed analysis of all classes in the system**, including **attributes, methods, relationships, multiplicity, and roles**. It is essentially the **preliminary design blueprint**.

Table 1:Detail Class with Attribute

Class	Attributes	Methods	Relationships	Multiplicity	Role
User	userID, name, email, phone, address	submitRequest(), viewStatus(), updateProfile()	WasteRequest	1..* WasteRequest per User	Citizen or organization generating waste requests
WasteRequest	requestID, wasteType, quantity, location, status, requestDate	createRequest(), updateStatus()	User, CollectionSchedule, RecyclingCenter	1..1 User, 0..1 CollectionSchedule	Represents a waste collection request
CollectionSchedule	scheduleID, staffID, vehicleID, date, time	assignRequest(), trackCollection()	WasteRequest, Staff, Vehicle	0..* WasteRequest , 1 Staff, 1 Vehicle	Manages scheduling of collections
Vehicle	vehicleID, licensePlate, capacity, status	assignCollection(), updateAvailability()	CollectionSchedule	0..* CollectionSchedule	Represents the vehicle used for waste collection
Staff	staffID, name, role, contact	performCollection(), updateRequestStatus()	CollectionSchedule, WasteRequest, Vehicle	1..* CollectionSchedule	Responsible for waste collection operations
RecyclingCenter	centerID, name, location,	receiveWaste(), updateProcessingStatus()	WasteRequest, Vehicle	0..* WasteRequest	Processes received waste

	capacity				
Report	reportID, type, date, generate dBy	generateReport(), exportReport()	WasteRequest, RecyclingCenter, Admin	0..*	Generates reports on system performance
Admin	adminID, name, role, contact	manageUsers(), manageVehicles(), generateSystemReport()	All classes	N/A	Manages system configurations, users, and reports

2.3.9 Logic Model

The logic model explains how the digital waste management system processes data by describing the step-by-step flow of actions for each major system process. It uses algorithms or pseudo code to clearly show how inputs are transformed into outputs through logical decision-making steps.

This section focuses on the core processes of the system: waste reporting, report verification, collection scheduling, and waste collection execution.

2.4 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements (NFRs) define **how the system performs** rather than **what the system does**. They focus on the **quality attributes** of the system such as performance, reliability, usability, and security.

For the Digital Waste Management and Recycling System, the key non-functional requirements are:

1. **Performance:**
 - The system should be able to handle **at least 1000 simultaneous waste collection requests** without significant delays.
 - Response time for user actions (e.g., submitting a request, generating a report) should be **under 3 seconds**.
2. **Scalability:**
 - The system must support **growth in users, vehicles, and collection data** without major redesign.
 - Must be able to integrate **future modules**, such as payment systems or recycling analytics.
3. **Reliability & Availability:**

- The system should be available **99% of the time**.
- Must include **backup and recovery mechanisms** to prevent data loss.
- 4. **Security:**
 - User authentication and role-based access control must be implemented (e.g., customers, employees, managers, admins).
 - Sensitive data (like personal customer information) must be **encrypted** in storage and transit.
 - Protection against common attacks such as **SQL injection, XSS, and unauthorized access**.
- 5. **Usability:**
 - The user interface should be **simple and intuitive** for all users.
 - Customers should be able to **request waste collection** in less than 2 minutes.
 - Staff and managers should have **clear dashboards** for tracking collections and generating reports.
- 6. **Maintainability:**
 - System code should be **modular and well-documented** to allow easy updates or bug fixes.
 - Ability to **update business rules** without rewriting the core system.
- 7. **Portability:**
 - The system should be **accessible via web browsers** on desktops, tablets, and smartphones.
 - Must work across common browsers (Chrome, Firefox, Edge).

2.5 SYSTEM REQUIREMENTS

The system requirements specify the **environment necessary for the Digital Waste Management and Recycling System to function correctly**. These requirements are divided into **hardware** and **software** categories. It is important to note that **exact specifications cannot be determined before system development and testing**, as they depend on factors such as the number of users, volume of data, and expected performance.

2.5.1 Hardware Requirements

- The hardware requirements describe the **type of equipment needed to run and access the system**:
 - Devices capable of running a web browser for customers and staff.
 - A server to host the system for handling requests, storing data, and generating reports.
 - Network infrastructure to enable communication between clients and the server.

Note: The exact specifications of the hardware (processor speed, RAM, storage, etc.) will be determined **after system development and performance evaluation**.

2.5.2 Software Requirements

- The software requirements describe the **programs and platforms necessary to develop, deploy, and use the system**:
 - An operating system that can support the backend and frontend technologies.
 - A database management system to store customer, employee, and collection data.
 - A web server or application server to handle system requests.
 - Development tools such as IDEs, libraries, and version control systems.
 - Web browsers for accessing the system interface.

Note: The exact versions or configurations of the software will be determined **based on the development tools chosen and compatibility with the system design**.

2.6 KEY ABSTRACTION WITH CRC ANALYSIS

The **CRC (Class-Responsibility-Collaborator) analysis** is used to identify the **key components of the digital waste management system**, define their **responsibilities**, and indicate how they **interact with each other**. This provides a clear structure for **system design** and ensures all functionalities are captured logically.

The following CRC table identifies the **main classes of the digital waste management system**, their **responsibilities**, and their **collaborators** to model the system’s logic.

Table 2: CRC (Class-Responsibility-Collaborator)

Class	Responsibilities	Collaborators
User	Submit waste collection requests, view request status, update profile	WasteRequest, Notification
WasteRequest	Store waste type, quantity, location, request status	User, CollectionSchedule, RecyclingCenter
CollectionSchedule	Assign collection requests to staff and vehicles, track collection progress	WasteRequest, Vehicle, Staff
Vehicle	Track availability and assignment for collections	CollectionSchedule, Staff
Staff	Collect waste according to schedule, update collection status	CollectionSchedule, Vehicle, WasteRequest

RecyclingCenter	Receive collected waste, update processing status	WasteRequest, Vehicle
Report	Generate reports on waste collection and recycling performance	WasteRequest, RecyclingCenter, Admin
Admin	Manage users, staff, vehicles, and system settings	All system classes

Notes:

- **User:** Represents citizens or organizations generating waste requests.
- **Staff & Vehicle:** Represent the resources used for collection and transportation of waste.
- **WasteRequest & CollectionSchedule:** Core system classes for managing collection operations.
- **RecyclingCenter & Report:** Handle waste processing and performance monitoring.
- **Admin:** Ensures system configuration, user management, and oversight.

This CRC analysis ensures that all **key functionalities of the digital waste management system** — from waste request submission to collection, transportation, and reporting — are clearly mapped to the appropriate system components.

2.6.1 Conceptual modeling: Class Diagram

A class diagram represents the static structure of the system by showing its classes, attributes, methods, and the relationships among them. It provides a clear view of how data is organized and how different components of the system interact at the structural level.

In the digital waste management system, the class diagram illustrates the main entities involved in waste collection and management, such as citizens, waste requests, municipal staff, waste collectors, schedules, and reports. These classes define how data is stored, accessed, and manipulated within the system.

The diagram helps in understanding system design, supports database schema development, and ensures that system components are well-structured, reusable, and scalable. By organizing responsibilities across different classes, the system eliminates manual record keeping and enables automated, data-driven waste collection and monitoring.

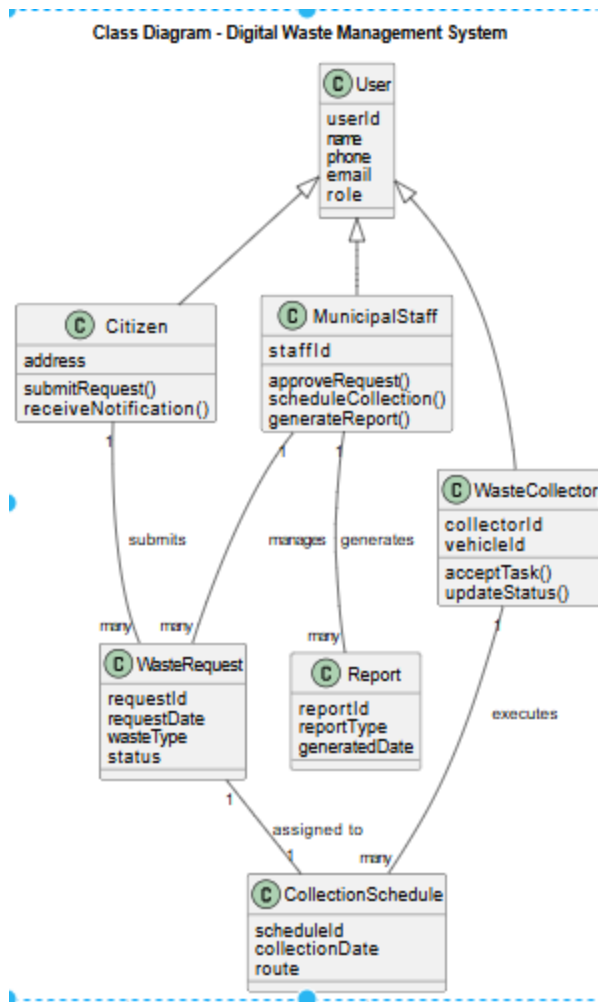


Figure 5:Class Diagram

2.6.2 Identifying Change Case

- In the **Digital Waste Management System**, a **change case** represents an **operation or process that changes the state of one or more classes** in the system. These are derived from the **class diagram and CRC analysis**, focusing on **dynamic state changes**.

Key Change Case:

1. Submit Waste Request

- Description:** When a user submits a request for waste collection, a new `WasteRequest` object is created, and the `CollectionSchedule` may be updated.
- Classes Involved:** `User`, `WasteRequest`, `CollectionSchedule`
- State Changes:**
 - `WasteRequest.status` → “Pending”
 - `CollectionSchedule.requests` → add new request

2. Assign Collection

- **Description:** The system assigns a staff member and a vehicle to a pending waste request.
- **Classes Involved:** `CollectionSchedule`, `Staff`, `Vehicle`
- **State Changes:**
 - `CollectionSchedule.assignedStaff` → updated
 - `CollectionSchedule.assignedVehicle` → updated
 - `Vehicle.status` → “Assigned”
- 3. **Update Collection Status**
 - **Description:** Staff updates the status of the collection after picking up waste.
 - **Classes Involved:** `Staff`, `WasteRequest`
 - **State Changes:**
 - `WasteRequest.status` → “Collected” or “In Transit”
- 4. **Deliver Waste to Recycling Center**
 - **Description:** When the waste is delivered, the `RecyclingCenter` updates records, and the `WasteRequest` status changes.
 - **Classes Involved:** `Vehicle`, `RecyclingCenter`, `WasteRequest`
 - **State Changes:**
 - `WasteRequest.status` → “Delivered”
 - `RecyclingCenter.receivedWaste` → updated
- 5. **Generate Report**
 - **Description:** The system creates performance reports based on collected and processed waste data.
 - **Classes Involved:** `Report`, `WasteRequest`, `RecyclingCenter`
 - **State Changes:**
 - New `Report` object created
 - `Report.content` → populated with analytics

2.6.3 User Interface Prototyping

The **User Interface (UI) Prototype** represents a **visual layout of the system**, showing how users will interact with the digital waste management system. Prototyping helps to **clarify functionality, improve usability, and gather feedback** before full implementation.

Key UI Components:

1. **Customer Interface**
 - **Home Page:** Overview of system services, quick links to submit requests.
 - **Waste Request Form:** Users can select waste type, quantity, and preferred collection time.
 - **Request Status Page:** Displays all pending, in-progress, and completed requests.
 - **Profile Management:** Allows users to update contact information and preferences.

2. Staff Interface

- **Dashboard:** Shows assigned collection schedules and vehicle assignments.
- **Collection Update Form:** Staff can update the status of each collection (e.g., “Collected,” “In Transit”).
- **Notification Panel:** Alerts for new assignments or schedule changes.

3. Admin Interface

- **User Management:** Add, edit, or deactivate users and staff.
- **Vehicle Management:** Track vehicles, assign schedules, and update status.
- **Recycling Center Overview:** Monitor waste delivery and processing status.
- **Reports:** Generate performance and operational reports.

Prototyping Tools

- Low-fidelity prototypes can be created using **wireframes** to visualize layout and navigation.
- High-fidelity prototypes can use tools like **Figma, Adobe XD, or Sketch**, showing detailed design, colors, and interactive elements.

Benefits of UI Prototyping:

- **Early Validation:** Stakeholders can see the system layout and provide feedback.
- **Improved Usability:** Helps to design interfaces that are intuitive for all user types.
- **Reduced Development Errors:** Provides a reference for developers to implement correct functionality.

Chapter 3: System Design

3. System Design Overview

This chapter describes the system design of the **Digital Demand-Based Waste Management and Recycling System**, which is proposed as a solution to the limitations of the existing waste management practices. The current system mainly depends on **manual operations and fixed-day collection schedules**, which do not adapt to changing waste generation patterns in urban areas. These limitations result in inefficient service delivery, poor resource utilization, delayed waste collection, and limited visibility for municipal authorities.

The purpose of the system design is to convert the identified problems, user needs, and system requirements into a **clear, structured, and implementable technical solution**. The design defines how system components are organized, how they interact, and how data flows across the system to support effective waste management operations. It acts as a bridge between the requirement analysis phase and the system implementation phase.

In the traditional waste management approach, collection routes and schedules are predetermined and remain unchanged regardless of actual waste demand. This often leads to **overflowing waste containers**

in high-density or high-demand areas, while collection vehicles still visit low-demand locations unnecessarily. Such inefficiencies increase operational costs, waste fuel and manpower, and negatively affect environmental cleanliness and public health.

The proposed system design addresses these challenges by introducing a **digital, demand-driven model**. Citizens actively participate in the process by reporting waste issues through the system, providing real-time data on waste generation. This data is processed by the system to generate **dynamic collection schedules and efficient resource management** based on actual demand, available vehicles, and workforce capacity. As a result, resources are allocated where they are most needed, improving efficiency and service quality.

Furthermore, the system design incorporates **centralized coordination and continuous monitoring**, allowing municipal staff to track waste collection activities in real time. Analytical and reporting features support **data-driven decision-making**, enabling authorities to evaluate performance, identify trends, and plan future improvements. By integrating these design elements, the system transforms waste management from a **reactive and schedule-based operation** into a **proactive, efficient, and sustainable digital service**.

3.1 Design Goals

The design objectives of the **Digital Demand-Based Waste Management and Recycling System** are derived from the problems identified in the existing waste management process and the goals of the proposed digital solution. These objectives guide the system architecture, component design, data modeling, and user interaction to ensure that the system is effective, sustainable, and suitable for real-world municipal use.

1. Replace Fixed-Day Waste Collection with Demand-Based Scheduling

One of the primary objectives of the system is to move away from the traditional fixed-day waste collection approach. In the existing system, waste is collected based on predetermined schedules rather than actual need, which often results in overflowing waste in some areas and unnecessary collection in others. The proposed system introduces demand-based scheduling, where waste collection activities are triggered by real-time reports from citizens and system-generated data. This ensures that collection is performed when and where it is needed most, improving service efficiency and cleanliness.

2. Improve Resource Utilization through Route, Vehicle, and Workforce Optimization

Efficient use of limited municipal resources is a key design objective. The system is designed to optimize collection routes, vehicle assignment, and workforce deployment based on demand and availability. By reducing unnecessary trips and balancing workloads, the system minimizes fuel consumption, operational costs, and manpower waste. This objective directly addresses inefficiencies caused by manual planning and supports environmentally and economically sustainable operations.

3. Support Real-Time Monitoring and Coordination among Stakeholders

Another important objective is to enable real-time monitoring and coordination between citizens, waste

collectors, municipal staff, and administrators. The system provides dashboards and notifications that allow stakeholders to track collection status, respond to issues promptly, and coordinate activities efficiently. This real-time visibility replaces fragmented communication and manual supervision with centralized digital control, improving transparency and accountability.

4. Enable Data-Driven Planning and Decision-Making

The system is designed to collect, store, and analyze waste management data over time. This objective ensures that municipal authorities can use historical and real-time data to generate reports, identify trends, and evaluate system performance. Data-driven insights support better planning of routes, resource allocation, policy decisions, and future system improvements, making waste management more strategic and informed.

5. Ensure Scalability, Security, Usability, and Maintainability

The final design objective focuses on long-term system quality. The system is designed to scale as the number of users, waste requests, and urban areas increase. Security mechanisms such as authentication and role-based access control protect sensitive data and system operations. Usability ensures that citizens and staff can easily interact with the system without extensive training, while maintainability ensures that the system can be updated, extended, and repaired efficiently over time. Together, these qualities ensure that the system remains reliable and adaptable in the long term.

In conclusion these design objectives ensure that the Digital Demand-Based Waste Management and Recycling System not only solves current operational problems but also provides a robust, flexible, and sustainable solution that delivers long-term operational and strategic value to municipalities.

the **web server, business logic, and database services**. The web server handles incoming requests from users and delivers dynamic web pages. The server processes waste requests, scheduling operations, notifications, and reporting functions, while the database server securely stores all system data such as users, waste requests, schedules, vehicles, and reports.

Communication between clients and servers takes place over **secure internet connections (HTTP/HTTPS)** to protect data privacy and system integrity. Centralized deployment allows municipal authorities to monitor operations in real time, enforce system policies consistently, and perform maintenance or updates without interrupting user access.

This deployment design supports **distributed access with centralized coordination**, which is essential for city-wide waste management operations. It also allows the system to scale easily by upgrading server resources or deploying cloud-based infrastructure as the number of users and data volume increase.

3.2 Architectural Design

3.1.1 Overview of the Architecture

The Digital Waste Management System follows a **web-based, three-tier (layered) architecture**. This architectural style separates the system into distinct layers, each with a specific responsibility. The separation improves scalability, maintainability, security, and ease of development.

The system is implemented using the following technologies:

- **Frontend:** React.js (client-side web application)
- **Backend:** Node.js with Express.js (RESTful API)
- **Database:** MySQL (relational database management system)

Users access the system through a web browser. Requests from the client are sent to the backend via RESTful APIs, processed according to business logic, and persisted in the MySQL database.

3.1.2 Architectural Layers

a) Presentation Layer (Frontend)

The presentation layer is implemented using **React.js**. It is responsible for handling all user interactions and displaying information to users in a clear and user-friendly manner.

Key responsibilities of the presentation layer include:

- Rendering user interfaces for different user roles (Citizen, Association Leader, Sub-City Admin, City Municipality Admin)
- Collecting user input such as waste collection requests, login credentials, and management actions
- Communicating with the backend server through RESTful API calls (HTTP requests)
- Displaying system responses, notifications, and status updates

The frontend does not directly access the database. All data access is performed through the backend API, which enhances security and control.

b) Application Layer (Backend / Business Logic Layer)

The application layer is implemented using **Node.js with Express.js**. This layer acts as an intermediary between the frontend and the database and contains the core business logic of the system.

Main responsibilities include:

- Exposing RESTful API endpoints for system operations
- Processing requests received from the frontend
- Applying business rules and validations
- Managing user authentication and authorization
- Handling role-based access control for different user types
- Coordinating data transactions with the database

The backend ensures that only authorized users can perform specific actions based on their assigned roles.

c) Data Layer (Database Layer)

The data layer uses **MySQL**, a relational database system, to store and manage all persistent data required by the system.

The database stores information such as:

- User accounts and roles
- Citizen waste collection requests
- Waste collection schedules and statuses
- Kebele, sub-city, and association data
- System logs and reports

Using a relational database ensures data consistency, integrity, and efficient querying.

3.1.3 System Interaction Flow

1. A user accesses the system through a web browser and interacts with the React-based frontend.
2. The frontend sends an HTTP request to the backend REST API.
3. The backend processes the request, applies business logic, and performs necessary validations.
4. The backend communicates with the MySQL database to retrieve or store data.
5. The processed response is sent back to the frontend.
6. The frontend updates the user interface accordingly.

3.1.4 Architectural Advantages

The chosen architecture provides several advantages:

- **Modularity:** Each layer can be developed and maintained independently.
- **Scalability:** The backend and database can be scaled to support more users and kebeles.
- **Security:** Direct database access is restricted to the backend only.
- **Maintainability:** Changes in one layer have minimal impact on others.
- **Flexibility:** The system can be extended with new features or services in the future.

This architectural design ensures that the Digital Waste Management System is robust, secure, and capable of supporting real-world municipal waste management operations.

3.1.5 Microservice-Oriented Architectural Approach

Although the system is currently deployed as a **single backend server**, the Digital Waste Management System is designed using a **microservice-oriented architectural approach at the logical level**. This means that the backend application is internally structured into independent and well-defined service modules, each responsible for a specific domain of functionality.

Key logical service modules include:

- Authentication and user management service
- Administrative management service (city and sub-city operations)
- Kebele and municipality rule management service
- Waste collection task management service
- Citizen request and feedback service
- Notification and communication service

Each service module encapsulates its own business logic and interacts with other modules through clearly defined interfaces. This modularization improves code organization, maintainability, and testability.

At the current stage, all service modules are deployed together within a single Node.js and Express.js application. However, the architecture is intentionally designed to allow future scalability. As the system grows in complexity or user load, individual service modules can be separated and deployed independently as standalone microservices without major redesign.

This hybrid approach combines the simplicity of monolithic deployment with the long-term scalability benefits of microservice architecture, making it suitable for municipal-level digital systems.

3.1.6 Communication Model

The system uses **HTTP/HTTPS-based communication** between the frontend and backend through RESTful APIs. All client requests and server responses follow standard HTTP methods such as GET, POST, PUT, and DELETE.

Real-time-like behavior is achieved using a **client-side polling mechanism** implemented in the frontend. The React application periodically sends lightweight requests to the backend at defined time intervals to check for updates such as:

- Changes in collection task status
- New notifications for associations
- Updates related to citizen requests

This polling-based communication approach provides near real-time updates while maintaining system simplicity and avoiding the complexity of persistent socket connections. It is well-suited for the current scale and operational requirements of the Digital Waste Management System.

3.2 Component Design

This section describes the **major system components** of the Digital Waste Management System and explains their responsibilities and interactions. Based on the documentation guideline and

best design practice, the component design is presented in a **combined manner**, covering both frontend and backend components, followed by the database component. This approach provides a clear end-to-end understanding of the system.

3.2.1 Frontend Component Design (React)

The frontend of the system is developed using **React.js** and follows a component-based architecture. Each component is responsible for a specific user interface or functionality, which improves reusability, maintainability, and clarity.

Major frontend components include:

- **Authentication Components**
Handle user login and access control for all user roles (Citizen, Association Leader, Sub-City Admin, City Municipality Admin).
- **Citizen Dashboard Components**
Allow citizens to submit waste collection requests, view request status, and participate in system conversations.
- **Association Leader Dashboard Components**
Enable association leaders to view assigned collection tasks, update task status, receive notifications, and communicate with administrators.
- **Sub-City Admin Dashboard Components**
Provide tools for monitoring kebeles, assigning associations, reviewing requests, and managing sub-city level operations.
- **City Municipality Admin Components**
Support city-level oversight, rule configuration, priority monitoring, and system-wide reporting.
- **Notification and Update Components**
Periodically fetch updates from the backend using polling mechanisms and display alerts and status changes.

3.2.2 Frontend Component Design (React)

Each frontend component communicates with the backend **exclusively through RESTful API endpoints** using HTTP/HTTPS. The frontend does not directly access the database. Data exchange is handled in JSON format, ensuring loose coupling between the presentation layer and the backend services.

The frontend also implements a **polling mechanism** using periodic requests (e.g., `setInterval`) to fetch updates such as task status changes, notifications, and new messages, enabling near real-time user experience.

3.2.3 Backend Component Design (Node.js and Express)

The backend of the Digital Waste Management System is developed using **Node.js and Express.js** and follows a **logically microservice-oriented, modular structure**. Each backend component is responsible for a specific business domain and exposes RESTful APIs for frontend interaction.

□ **Major backend components include:**

- **Authentication and Authorization Component**
This component handles user authentication, password verification, and role-based access control. It ensures that only authorized users can access system functionalities based on their assigned roles.
 - **User and Role Management Component**
Manages user accounts and role-specific profiles, including citizens, association leaders, sub-city administrators, and city municipality administrators.
 - **Administrative Management Component**
Handles city, sub-city, kebele, and municipality rule management. This component supports administrative configuration and system governance activities.
 - **Waste Collection Task Management Component**
Responsible for generating, assigning, updating, and tracking waste collection tasks. It applies priority rules and manages task lifecycle states such as pending, assigned, in-progress, and completed.
 - **Citizen Request Management Component**
Processes waste collection requests submitted by citizens, maintains request status, and supports request expiration and tracking.
 - **Notification and Communication Component**
Manages notifications sent to associations and supports structured communication through conversations and messages between citizens, associations, and administrators.
- ❖ Although these components are currently deployed within a single backend application, they are designed to be **independent and loosely coupled**, allowing future separation into standalone microservices if required.

3.2.4 Database Component Design

- The database component is implemented using **MySQL**, a relational database management system. It provides persistent data storage and ensures data consistency, integrity, and reliability.

The database stores and manages:

- Administrative hierarchy data (cities, sub-cities, kebeles)
- Municipality rules used for priority calculation
- User accounts and role-specific entities
- Waste collection tasks and their status
- Citizen requests and feedback

- Notifications and inter-user conversations
- ❖ The backend components interact with the database through controlled queries, enforcing referential integrity through foreign key constraints and preventing unauthorized data access.

3.2.5 Component Interaction Summary

Frontend components initiate requests to backend service components via RESTful APIs. Backend components process these requests, apply business logic, and interact with the database component to retrieve or store data. The processed responses are then returned to the frontend, which updates the user interface accordingly.

This component-based design enhances **modularity, scalability, and maintainability** of the Digital Waste Management System.

THE COMPONENT DIAGRAM

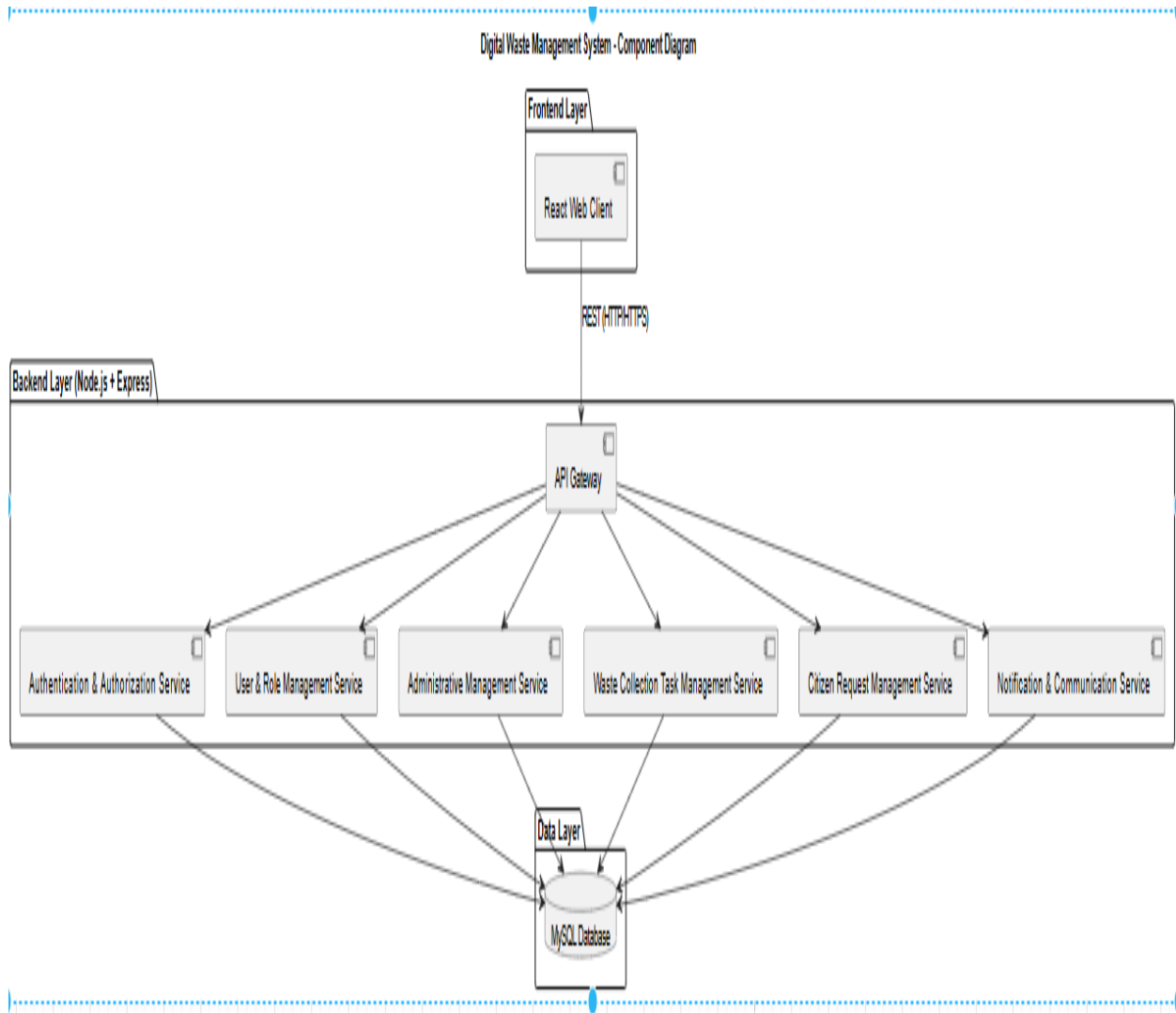


Figure 6: Component Diagram

3.1.7 Deployment Modeling

- Deployment modeling describes the physical execution environment of the Digital Waste Management System. While the component design focused on the logical software structure, the deployment model defines how these software artifacts are distributed across hardware nodes, including the communication protocols that

- facilitate data exchange between them.
- The system utilizes a distributed **3-Tier Architecture**, which provides high scalability, security, and ease of maintenance by separating the user interface, the application logic, and the data persistence layers into distinct execution environments.

Description of Deployment Nodes

1. **Client Node (Presentation Layer)** The Client Node represents the end-user's environment, typically a web browser on a personal computer or a mobile device.
 - **Artifacts:** The React.js frontend application is served as a set of static assets (HTML, CSS, JavaScript).
 - **Functionality:** It handles the rendering of the user interface and captures user inputs. It does not contain business logic; instead, it communicates with the application server via asynchronous requests.
2. **Application Server Node (Logic Layer)** This node represents the server-side environment where the Node.js runtime environment is installed.
 - **Artifacts:** The Express.js application, which includes all the service components (Auth, Task Management, etc.) previously defined in the component design.
 - **Functionality:** It serves as the "brain" of the system. It receives RESTful API requests from the Client Node, performs authentication and authorization, applies municipality rules for priority calculations, and coordinates data flow.
 - **Execution Environment:** Managed through a process manager (like PM2) to ensure high availability and automatic restarts in case of failure.
3. **Database Server Node (Data Layer)** The Data Layer is hosted on a dedicated database server running the MySQL Relational Database Management System (RDBMS).
 - **Artifacts:** The Physical Schema (Tables, Views, and Constraints) defined in the SQL schema.
 - **Functionality:** It handles persistent storage, ensures data integrity through foreign key constraints, and manages concurrent data access. Isolating the database on a separate node enhances security, as it can be configured to only accept incoming connections from the Application Server's IP address.

Communication Protocols and Connections

- **Client to Application Server:** Communication is performed over **HTTP/HTTPS** using the **REST (Representational State Transfer)** architectural style. All data is exchanged in **JSON** format.
- **Application Server to Database Server:** The connection is established via the **MySQL Protocol** (typically on port 3306). The backend uses a connection pooling mechanism to optimize performance and manage multiple simultaneous queries efficiently.

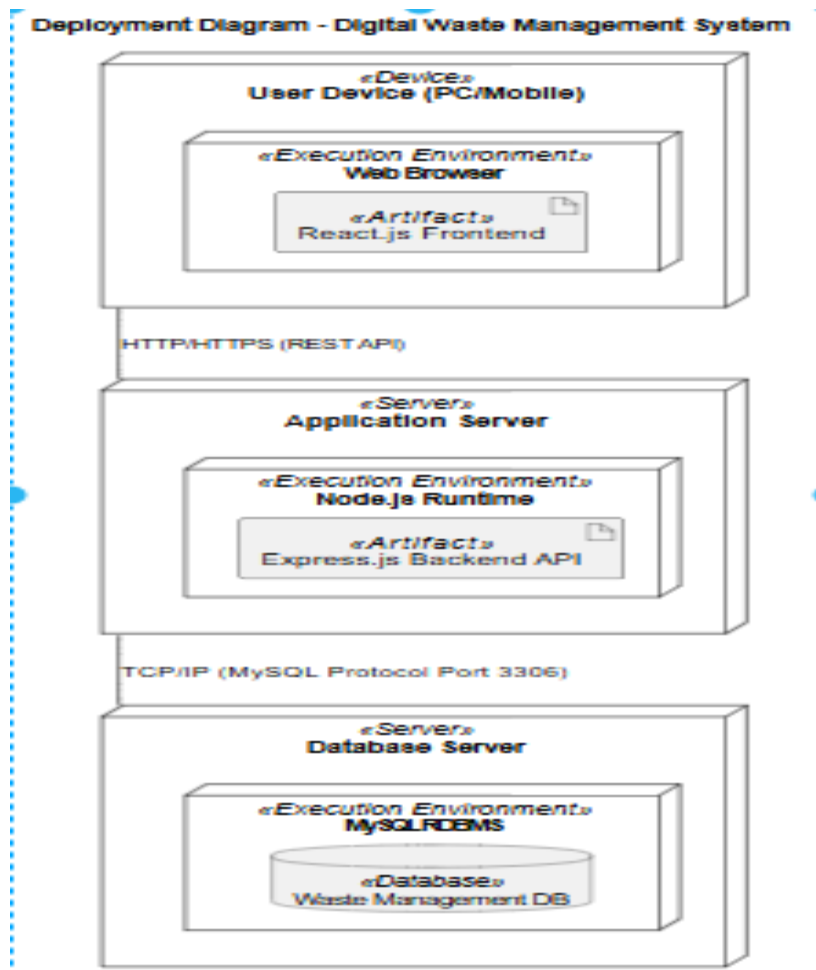


Figure 7:Deployment Diagram

3.2 Detail Design

- This section presents the detailed design of the **Digital Demand-Based Waste Management and Recycling System**. It focuses on the internal structure of the system, including the design class model and the persistent data model. The detailed design translates the architectural design

3.2.1 Design Class Model

- The Design Class Model for the Digital Waste Management System follows an Object-Oriented approach where each class is defined with its specific attributes, return types, and arguments.

1. Administrative and User Classes

- **User (Entity)**
 - **Attributes:** -user_id: int, -email: String, -password_hash: String, -role: Enum, -created_at: Timestamp.

- **Methods:** +verifyPassword(pwd: String): boolean, +getRole(): Enum.
- **CityAdmin / SubCityAdmin / Leader (Entities)**
 - **Attributes:** -id: int, -user_id: int, -first_name: String, -last_name: String, -assigned_id: int.
 - **Methods:** +getFullName(): String.
- **UserController (Logic)**
 - **Methods:** +registerUser(data: Object): Response, +login(credentials: Object): Token, +getProfile(id: int): Object.

2. Infrastructure and Governance Classes

- **City / SubCity / Kebele (Entities)**
 - **Attributes:** -id: int, -name: String, -parent_id: int, -population: int, -waste_rate: float, -max_capacity: float.
 - **Methods:** +getHierarchy(): String, +calculateCurrentLoad(): float.
- **MunicipalityRule (Entity)**
 - **Attributes:** -rule_id: int, -kebele_type: String, -threshold_percent: float, -area_importance_weight: float.
 - **Methods:** +getWeight(): float.
- **AdminService (Service)**
 - **Methods:** +addKebele(data: Object): int, +updateRules(ruleId: int, data: Object): boolean.

3. Operational and Task Management Classes

- **Association (Entity)**
 - **Attributes:** -association_id: int, -name: String, -truck_count: int, -worker_count: int, -sub_city_id: int.
 - **Methods:** +isAvailable(): boolean, +updateResources(trucks: int, workers: int): void.
- **CollectionTask (Entity)**
 - **Attributes:** -task_id: int, -kebele_id: int, -association_id: int, -priority_score: float, -status: Enum.
 - **Methods:** +calculatePriority(): void, +changeStatus(newStatus: Enum): boolean.
- **TaskService (Service)**
 - **Methods:** +generateTasks(): List<Task>, +assignToAssociation(taskId: int, assocId: int): void.

4. Citizen Interaction and Communication Classes

- **Citizen (Entity)**
 - **Attributes:** -citizen_id: int, -user_id: int, -kebele_id: int, -phone: String.
- **CollectionRequest (Entity)**
 - **Attributes:** -request_id: int, -citizen_id: int, -request_text: String,

- status: Enum.
 - o **Methods:** +expireRequest(): void.
- **Conversation / Message (Entities)**
 - o **Attributes:** -id: long, -type: Enum, -sender_id: long, -text: String, -status: Enum.
 - o **Methods:** +sendMessage(): boolean, +archiveConversation(): void.
- **Notification (Entity)**
 - o **Attributes:** -notification_id: int, -association_id: int, -kebele_id: int, -message: String.

sign into concrete software elements that guide system implementation.

Class Relationships and Multiplicity

- **User (1) → (0..*) WasteRequest**
A user can submit multiple waste requests.
- **WasteRequest (1) → (1) CollectionSchedule**
Each request is assigned to a collection schedule.
- **CollectionSchedule (1) → (1..*) Vehicle / Staff**
A schedule can involve multiple vehicles and staff members.
- **CollectionSchedule (1) → (1) RecyclingCenter**
- **Municipal Staff (1) → (0..*) Report**
- ❖ All attributes are **private** to enforce encapsulation, while methods are **public** to allow controlled access.

3.2.2 Persistent Model

- Since this system utilizes a **Relational Database (MySQL)**, the guideline requires an **E-R Diagram** to show the mappings and relations of the tables. The persistent model below defines the specific attributes, data types, lengths, and the referential integrity maintained through keys.

I. Entity-Relationship (E-R) Diagram

Below is the PlantUML code for the **E-R Diagram** representing your 15-table relational schema.

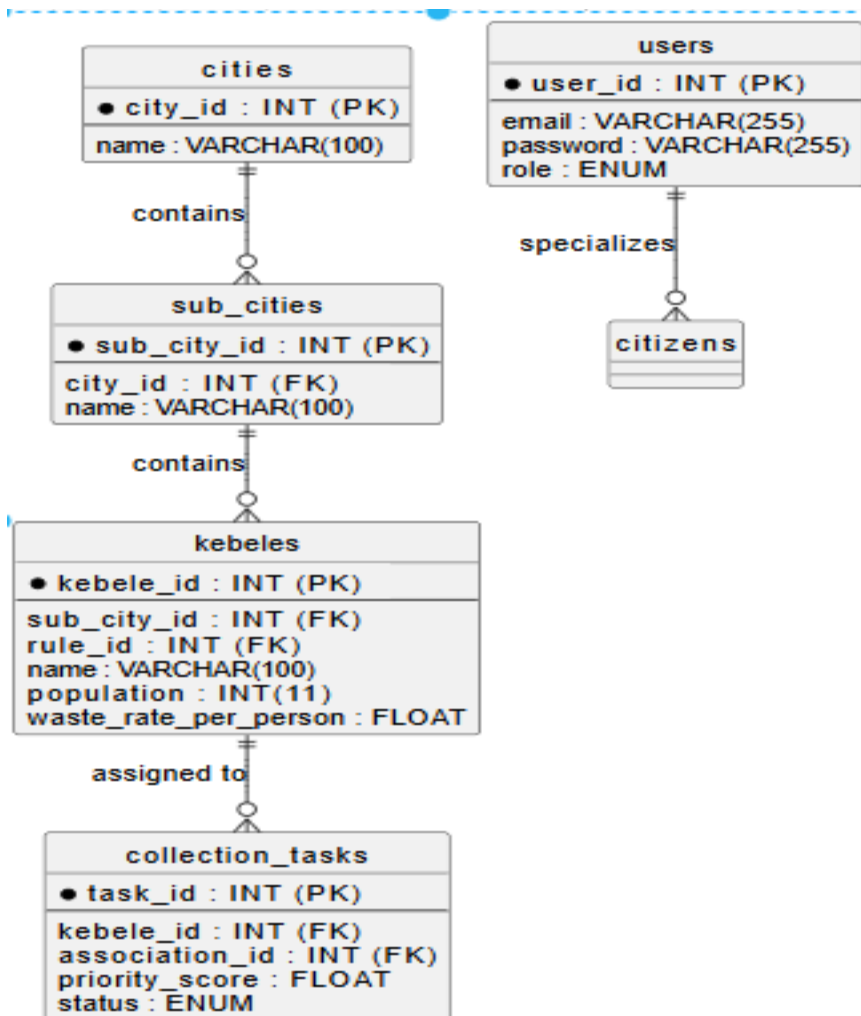


Figure 8:E-R Diagram

Persistent Tables and Mappings

Table 3:Persistent Tables and Mappings

Class	Table Name	Primary Key
User	users	user_id
WasteRequest	waste_requests	request_id
CollectionSchedule	collection_schedules	schedule_id
Vehicle	vehicles	vehicle_id

Staff	staff	staff_id
RecyclingCenter	recycling_centers	center_id
Report	reports	report_id

Key Relationships (Foreign Keys)

- waste_requests.user_id → users.user_id
- waste_requests.schedule_id → collection_schedules.schedule_id
- collection_schedules.vehicle_id → vehicles.vehicle_id
- collection_schedules.staff_id → staff.staff_id
- collection_schedules.center_id → recycling_centers.center_id

Persistent Model Characteristics

- **Primary keys** uniquely identify each record.
 - **Foreign keys** enforce relationships between tables.
 - **One-to-many relationships** support real-world workflows (e.g., one user → many requests).
 - **Normalization** reduces redundancy and improves performance.
- ❖ This persistent model supports **reliable analytics, reporting, auditing, and decision-making**, which are essential for demand-based waste management operations.

3.3 User Interface Design

- **The user interface of the Digital Demand-Based Waste Management and Recycling System is designed to effectively support the needs of different stakeholders, including citizens, waste collectors, municipal staff, and system administrators. Since the system is web-based, the interface plays a critical role in ensuring usability, accessibility, and user satisfaction.**
- For **citizens**, the interface provides a simple and intuitive way to report waste issues, submit collection requests, and track the status of their requests. Clear forms, status indicators, and notifications help citizens interact with the system easily, encouraging active participation in maintaining a clean environment.
- For **waste collectors**, the user interface displays assigned collection routes, schedules, and task details. Collectors can update task completion status in real time, allowing the system to reflect current operational progress. This reduces the need for manual reporting and improves

coordination with municipal staff.

- For **municipal staff**, the interface offers dashboards for monitoring waste collection activities, reviewing system performance, and generating analytical reports. Visual summaries and filtering options help staff identify problem areas, evaluate efficiency, and make informed operational decisions.
- For **administrators**, the user interface provides tools to manage system users, roles, permissions, and configuration settings. Administrative functions ensure smooth system operation, security enforcement, and policy management.
- UI prototyping is used during the design phase to validate layouts, workflows, and usability before implementation. This approach ensures that the system is intuitive, reduces user training requirements, and supports efficient daily operations across all user groups.

3.3.1 Authentication and Landing Interface

- This is the entry point for all users. It serves as the gateway to the system's internal functionalities.
- **Design Logic:** A centralized login screen that identifies the user's role upon successful authentication and redirects them to their specific dashboard.
- **Key Elements:** Email/Username input fields, password field with visibility toggle, and "Forgot Password" functionality.
- **Layout:** A clean, centered form card with the municipality logo to establish trust and branding.

LOGIN PAGE

The screenshot displays the login interface of the Digital Waste Management System (DWMS). The header includes the DWMS logo and navigation links for Home, About, and Contact. On the right side of the header, there are icons for a globe, a dark mode toggle, and a 'Login' button, along with a green 'Sign Up' button. The main content area features a centered 'Login' form. This form contains an 'Email' input field with the text 'direshibritu@gmail.com', a 'Password' input field with masked characters '*****', and a green 'Login' button. Below the password field, there are links for 'Don't have an account? Sign up' and 'Forgot your password?'. The footer is divided into three sections: 'DWMS' with a description of the system, 'Quick Links' with links to Home, About Us, and Contact, and 'Contact' with email and phone information. A copyright notice for 2025 is at the bottom.

Figure 9: Login Page

3.3.2 Citizen(Customer) Interface (Request & Signal Portal)

This interface is designed for simplicity, prioritizing the "Signal" or "Request" feature which is the primary touchpoint for residents.

- **Waste Reporting Form:** A simple interface where citizens can select their waste type and submit a "Full Bin" signal.
- **Request Tracking Table:** Shows the status of the citizen's current requests (e.g., Pending, Scheduled, or Completed).
- **Notification Bell:** A persistent UI element that alerts the citizen when an association is dispatched to their Kebele.
- **Design Logic:** Mobile-first design to ensure citizens can report waste easily using smartphones.

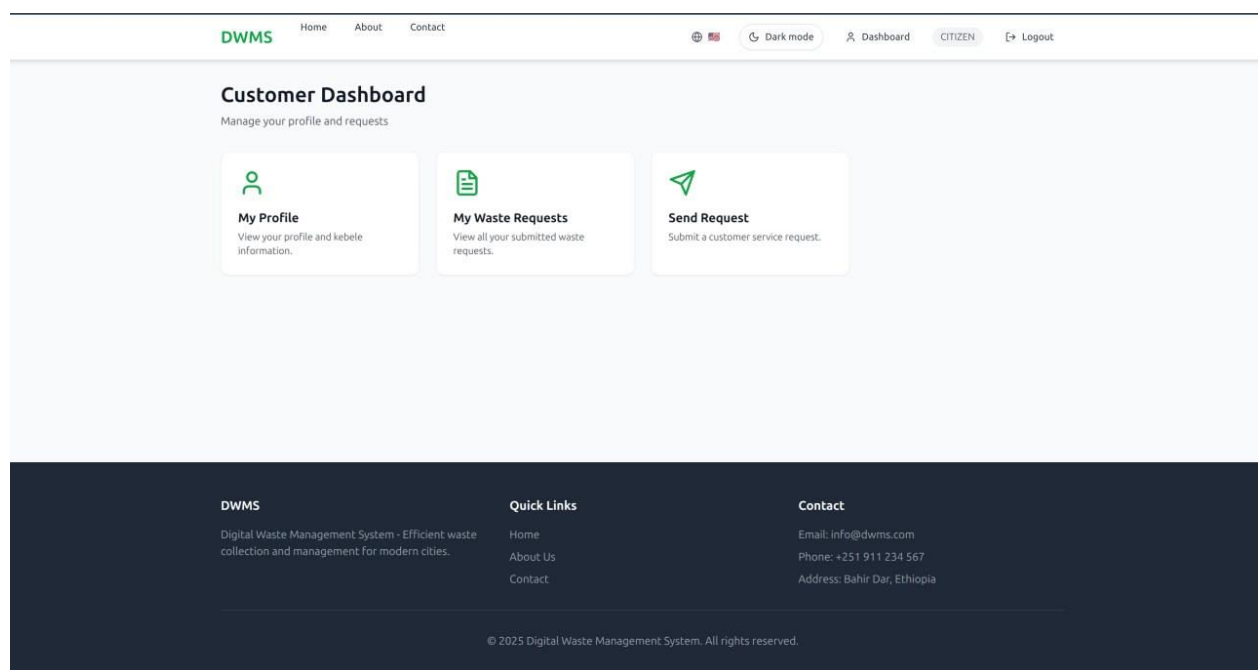


Figure 10: Citizen Dashboard

3.3.3 Association Leader Interface (Operational View)

This interface is highly functional, focusing on task execution and communication with citizens.

- **Task Assignment List:** Displays all collection tasks assigned to the specific association by the Sub-City Admin.
- **Notification Dispatcher:** A specific interface where the leader can click "Notify Kebele" to trigger automated alerts to all citizens in a target area before the truck arrives.
- **Resource Management:** Fields to update the active number of trucks and personnel

available for the day.

- **Design Logic:** High-contrast data tables to allow leaders to quickly scan for "Urgent" or "High Priority" tasks.

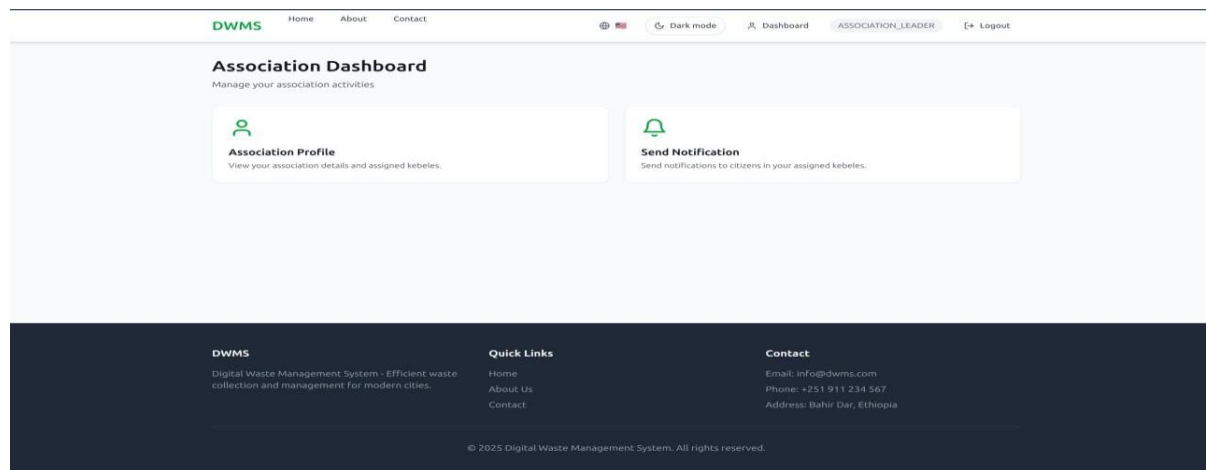


Figure 11: Association Leader Dashboard

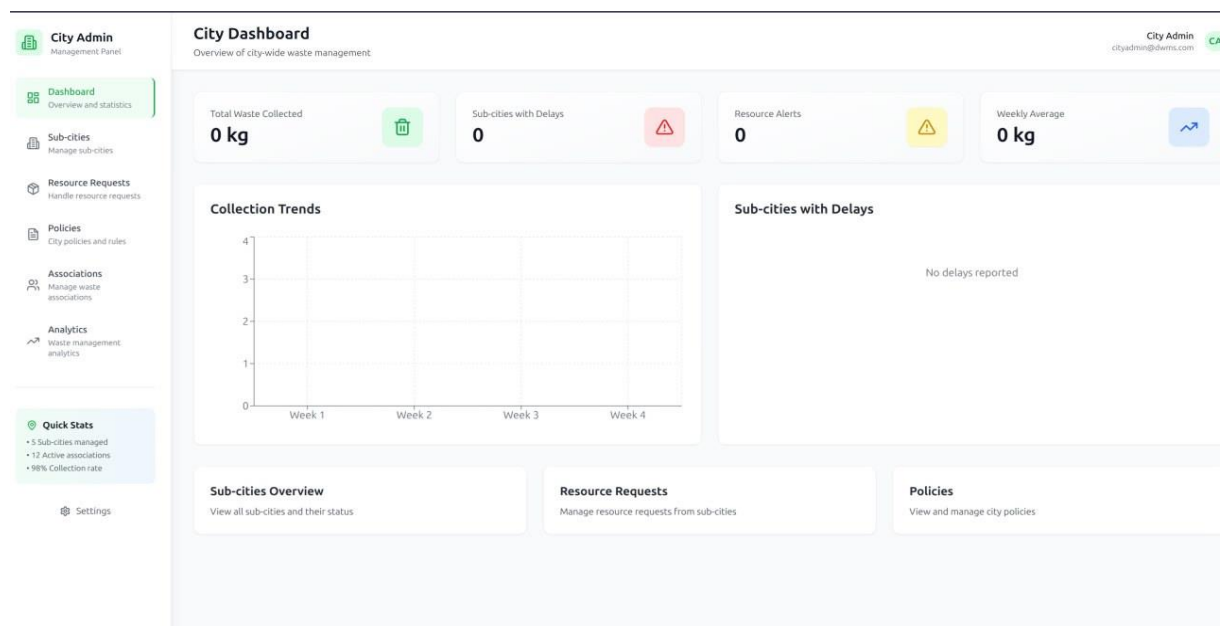


Figure 12 :City Admin Dashboard

3.3.5 Sub-City Admin Dashboard

The most complex interface, designed for high-level monitoring and governance.

- **Kebele Monitoring Grid:** A visual grid showing all Kebeles under the admin's jurisdiction, color-coded by waste level (e.g., Red for >80% capacity).

- **Rule Configuration Panel:** An interface where admins can adjust the `threshold_percent` and `area_importance_weight` to fine-tune the priority calculation algorithm.
- **Task Generation Hub:** A manual and automated interface to generate collection tasks based on aggregated citizen signals.
- **Design Logic:** A sidebar-based navigation system to allow quick switching between Kebele management, User management, and Reports.

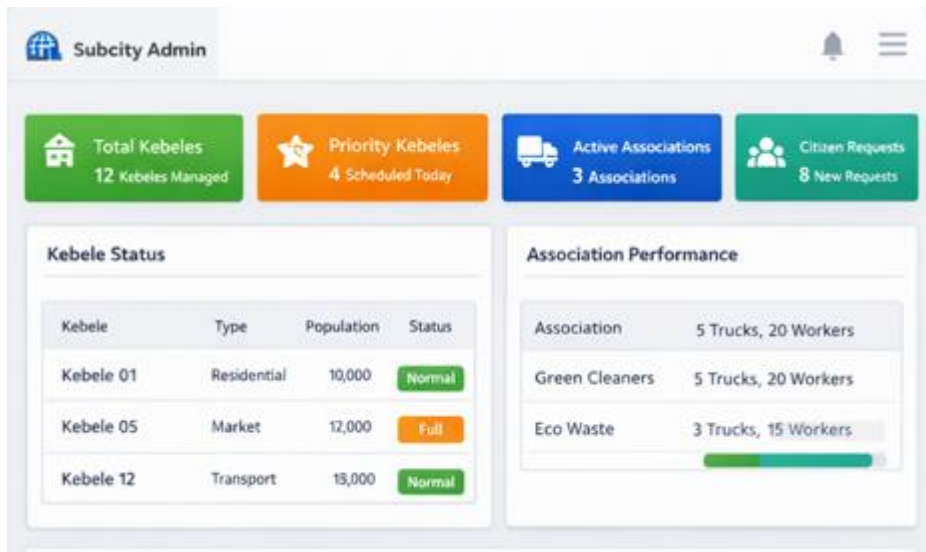


Figure 13:Sub-City Admin Dashboard

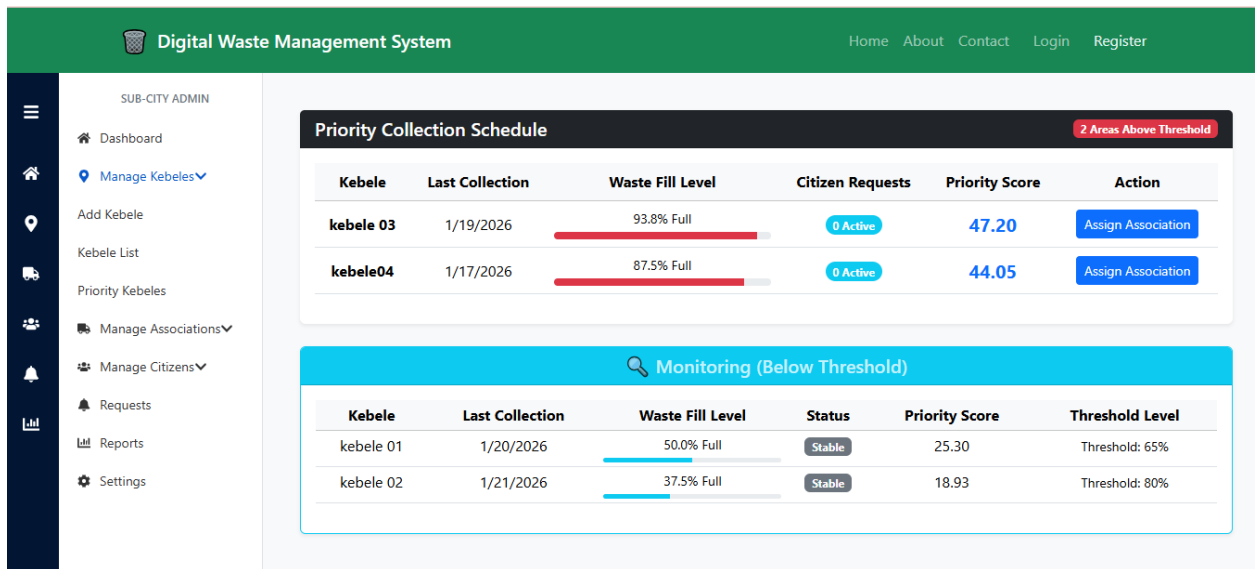


Figure 14:Sub-City kebeles page

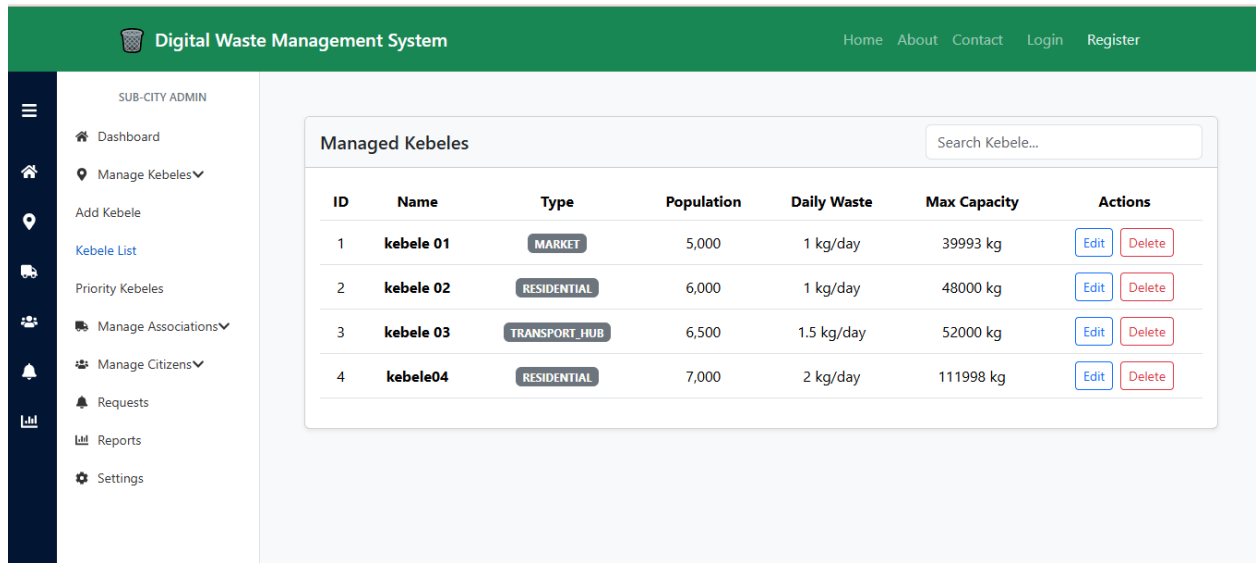


Figure 15: Sub-City Admin kebeles list page

The UI design follows usability principles such as **consistency, clarity, feedback, and responsiveness**, reducing training requirements and supporting efficient daily operations.

3.6 Access Control and Security

Access control and security are critical aspects of the Digital Demand-Based Waste Management and Recycling System, as the system manages sensitive operational and user data. The system implements **role-based access control (RBAC)** to ensure that each actor can access only the functions and data relevant to their role.

User Roles and Access Levels

- **Citizens**

- Register and log in
- Submit waste collection requests
- View request status and notifications
- Cannot access administrative or operational data

- **Waste Collectors**

- View assigned schedules and routes
- Update collection status
- Cannot modify system configurations or reports

- **Municipal Staff**

- Monitor system operations
- View and generate reports
- Manage schedules and assignments
- Limited access to system configuration

- **Administrators**

- Full access to all system functions
 - Manage users, roles, and permissions
 - Configure system settings
 - Monitor security activities

Security Mechanisms

- **Authentication:**
Users must log in using valid credentials before accessing the system.
- **Authorization:**
Access to system features is restricted based on user roles.
- **Data Protection:**
Sensitive data is securely stored and accessed through controlled database operations.
- **Secure Communication:**
Data transmission between users and servers occurs over secure internet connections (HTTPS).

These access control and security measures ensure **confidentiality, integrity, and availability** of system data while preventing unauthorized access and misuse.

Chapter Four: Implementation

4.1 Implementation Overview

This chapter presents the implementation of the **Digital Demand-Based Waste Management and Recycling System**, translating the system design into a functional **web-based application**. The implementation follows the architectural and detailed design specifications defined in Chapter 3, ensuring that all functional and non-functional requirements are properly realized.

The system is implemented using **modern web technologies** with a clear separation between frontend and backend components. The frontend provides interactive and user-friendly interfaces for different stakeholders, while the backend manages business logic, data processing, security, and communication with the database.

The implementation adopts a **modular and layered approach**, where system functionalities are divided into reusable components and services. This structure improves code readability, maintainability, and scalability. Core features such as waste request submission, demand-based scheduling, route assignment, real-time status updates,

notifications, and reporting are implemented as independent modules that interact through well-defined interfaces.

Security and access control mechanisms are integrated during implementation to ensure that only authorized users can access specific system functions. Role-based access control, authentication, and input validation are enforced at the backend level to protect system data and operations.

The system is developed incrementally and tested throughout the implementation process. This approach allows early detection of errors and ensures that individual components function correctly before full system integration. Overall, the implementation results in a reliable, scalable, and secure web-based system that supports efficient, demand-driven waste management operations.

The system supports multiple roles:

- ✓ **Admin**
 - ✓ **City Admin**
 - ✓ **Sub-city Admin**
 - ✓ **Association Leader**
 - ✓ **Citizen**
- Each role has restricted access to specific APIs and pages, ensuring system security.

4.2 Backend Implementation

The backend of the Digital Demand-Based Waste Management and Recycling System is implemented using **Node.js** with the **Express.js framework**, following a **RESTful API architecture**. This technology stack is chosen due to its scalability, performance, and suitability for building web-based systems that support multiple concurrent users and real-time operations.

Node.js provides a non-blocking, event-driven runtime environment that efficiently handles multiple client requests, which is essential for a municipal system accessed by citizens, waste collectors, and administrative staff. Express.js is used to simplify server-side development by providing routing, middleware support, and structured request handling.

The following implementation code are only sample

4.2.1 Admin

```
//admin seeder
```

```

require("dotenv").config();
const bcrypt = require("bcrypt");
const pool = require("../config/db");

const ADMIN_EMAIL = "admin@dwms.com";
const ADMIN_PASSWORD = "Admin@123";
const ADMIN_NAME = "System Admin";
const ADMIN_ROLE = "ADMIN";

async function seedAdmin() {
  console.log(" ADMIN SEEDER FILE IS EXECUTING");

  try {
    const hashedPassword = await bcrypt.hash(ADMIN_PASSWORD, 10);

    const query = `
      INSERT INTO users (name, email, password_hash, role, is_verified)
      VALUES (?, ?, ?, ?, true)
      ON DUPLICATE KEY UPDATE
        name = VALUES(name),
        password_hash = VALUES(password_hash),
        role = VALUES(role),
        is_verified = true
    `;

    await pool.query(query, [
      ADMIN_NAME,
      ADMIN_EMAIL,
      hashedPassword,
      ADMIN_ROLE,
    ]);

    console.log(" Admin user seeded successfully:", ADMIN_EMAIL);
  } catch (err) {
    console.error(" Failed to seed admin user:", err.message);
  } finally {
    process.exit(0);
  }
}

seedAdmin();

```

4.2.2 Authentication and authorization

```

const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const pool = require("../config/db");

```

```

const sendEmail = require("../utils/sendEmail");

// Allowed roles for registration (Admin is seeded separately)
const ALLOWED_ROLES = ["Citizen", "Sub-City Admin", "City-Admin", "Association leader"];

const register = async (req, res) => {
  try {
    const { name, email, password, role } = req.body;

    // Trim and normalize role
    const normalizedRole = role ? role.trim() : "";

    // Debug logging
    console.log("Registration attempt - Role received:", JSON.stringify(role));
    console.log("Normalized role:", JSON.stringify(normalizedRole));
    console.log("Allowed roles:", ALLOWED_ROLES);

    if (!normalizedRole || !ALLOWED_ROLES.includes(normalizedRole)) {
      return res.status(400).json({
        error:
          "Invalid role. Allowed roles are Citizen, Sub-City Admin, City-Admin, Association leader.",
        received: normalizedRole,
        allowed: ALLOWED_ROLES
      });
    }

    const hashed = await bcrypt.hash(password, 10);
    const otp = Math.floor(100000 + Math.random() * 900000);
    const expiry = new Date(Date.now() + 10 * 60000);

    await pool.query(
      `INSERT INTO users
      (name, email, password, role, otp_code, otp_expires)
      VALUES (?, ?, ?, ?, ?, ?)`,
      [name, email, hashed, normalizedRole, otp, expiry]
    );

    // Try to send email, but don't fail registration if email fails
    try {
      await sendEmail(
        email,
        "DWMS Email Verification",
        `Your OTP code is ${otp}. It expires in 10 minutes.`
      );
      console.log(`☐ OTP email sent to ${email}`);
    } catch (emailError) {
      console.error("☐ Email sending failed (registration still successful):", emailError.message);
      // Log OTP to console for development/testing
      console.log(`☐ OTP for ${email}: ${otp} (Email service unavailable)`);
    }
  }
}

```

```

res.json({
  message: "Registration successful. Please check your email for OTP code.",
  // Include OTP in response for development (remove in production)
  ...(process.env.NODE_ENV !== 'production' && { otp: otp })
});
} catch (err) {
  console.error("REGISTER ERROR:", err);

  if (err.code === "ER_DUP_ENTRY") {
    return res.status(409).json({ error: "Email already registered" });
  }

  res.status(500).json({ error: err.message || "Internal server error" });
}
};

const verifyOtp = async (req, res) => {
  try {
    const { email, otp } = req.body;

    if (!email || !otp) {
      return res.status(400).json({ error: "Email and OTP are required" });
    }

    const [rows] = await pool.query("SELECT * FROM users WHERE email = ?", [email]);

    if (!rows.length)
      return res.status(404).json({ error: "User not found" });

    const dbUser = rows[0];

    // Convert both to strings for comparison (handles INT from DB and string from request)
    const receivedOtp = String(otp).trim();
    const dbOtp = String(dbUser.otp_code).trim();

    // Check if OTP matches
    if (receivedOtp !== dbOtp) {
      console.log("❌ OTP mismatch:", { received: receivedOtp, stored: dbOtp });
      return res.status(400).json({ error: "Invalid OTP code" });
    }

    // Check if OTP is expired
    const now = new Date();
    const expiryDate = new Date(dbUser.otp_expires);

    if (now > expiryDate) {
      return res.status(400).json({ error: "OTP has expired. Please request a new one." });
    }
  }
};

```

```

// Check if user is already verified
if (dbUser.is_verified) {
  return res.status(400).json({ error: "Email is already verified" });
}

// Verify the user
await pool.query(
  "UPDATE users SET is_verified=true, otp_code=NULL, otp_expires=NULL WHERE email=?",
  [email]
);

console.log(" ☐ OTP verified successfully");
res.json({ message: "Email verified successfully" });
} catch (err) {
  console.error("VERIFY OTP ERROR:", err);
  res.status(500).json({ error: err.message || "Internal server error" });
}
};

const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    const [rows] = await pool.query("SELECT * FROM users WHERE email=?", [
      email,
    ]);

    if (!rows.length)
      return res.status(404).json({ error: "User not found" });

    const user = rows[0];

    if (!user.is_verified)
      return res.status(403).json({ error: "Email not verified" });

    const valid = await bcrypt.compare(password, user.password);
    if (!valid)
      return res.status(401).json({ error: "Invalid password" });

    const token = jwt.sign(
      { id: user.id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: "1d" }
    );

    res.json({ token });
  } catch (err) {
    console.error("LOGIN ERROR:", err);
    res.status(500).json({ error: err.message || "Internal server error" });
  }
}

```

```

};

const resendOtp = async (req, res) => {
  try {
    const { email } = req.body;

    if (!email) {
      return res.status(400).json({ error: "Email is required" });
    }

    const [rows] = await pool.query("SELECT * FROM users WHERE email = ?", [email]);

    if (!rows.length) {
      return res.status(404).json({ error: "User not found" });
    }

    const user = rows[0];

    // Check if user is already verified
    if (user.is_verified) {
      return res.status(400).json({ error: "Email is already verified" });
    }

    // Generate new OTP
    const otp = Math.floor(100000 + Math.random() * 900000);
    const expiry = new Date(Date.now() + 10 * 60000); // 10 minutes from now

    // Update OTP in database
    await pool.query(
      "UPDATE users SET otp_code=?, otp_expires=? WHERE email=?",
      [otp, expiry, email]
    );

    // Try to send email, but don't fail if email fails
    try {
      await sendEmail(
        email,
        "DWMS Email Verification - New OTP",
        `Your new OTP code is ${otp}. It expires in 10 minutes.`
      );
    } catch (emailError) {
      console.error("❌ Email sending failed (OTP still updated):", emailError.message);
      // Log OTP to console for development/testing
      console.log(`❌ New OTP for ${email}: ${otp} (Email service unavailable)`);
    }

    res.json({
      message: "New OTP sent to email. Please verify.",
      // Include OTP in response for development (remove in production)
      ...(process.env.NODE_ENV !== 'production' && { otp: otp })
    });
  }
};

```

```

    });
  } catch (err) {
    console.error("RESEND OTP ERROR:", err);
    res.status(500).json({ error: err.message || "Internal server error" });
  }
};

const ping = (req, res) => {
  res.json({ message: "Swagger is working ☑" });
};

module.exports = {
  register,
  verifyOtp,
  resendOtp,
  login,
  ping,
};

```

Association Controller

```

const pool = require("../config/db");

// Get association profile
const getProfile = async (req, res) => {
  try {
    const userId = req.user.id;

    // Check if user is approved
    const [userRows] = await pool.query(
      "SELECT id, is_approved FROM users WHERE id = ?",
      [userId]
    );

    if (!userRows.length) {
      return res.status(404).json({
        success: false,
        error: "User not found"
      });
    }

    const user = userRows[0];

    // Check approval status
    if (!user.is_approved) {
      return res.status(403).json({
        success: false,
        error: "Your account is pending approval. Please wait for admin approval before accessing your profile.",
        pending_approval: true
      });
    }
  }
};

```

```

}

// Get association info
const [assocRows] = await pool.query(
  `SELECT a.id, a.name, a.sub_city
  FROM associations a
  WHERE a.leader_id = ?
  LIMIT 1`,
  [userId]
);

if (!assocRows.length) {
  // Return empty profile if no association yet (but user is approved)
  return res.json({
    success: true,
    data: {
      association_name: null,
      assigned_kebeles: [],
      tasks_today: 0,
      message: "No association found. Please contact an administrator to create an association for your
account."
    }
  });
}

const association = assocRows[0];

// Get assigned kebeles
const [kebeleRows] = await pool.query(
  `SELECT k.id, k.name, ak.status
  FROM association_kebeles ak
  JOIN kebeles k ON ak.kebele_id = k.id
  WHERE ak.association_id = ?`,
  [association.id]
);

const assignedKebeles = kebeleRows.map(k => ({
  kebele_id: k.id,
  kebele_name: k.name,
  current_status: k.status || "ASSIGNED"
}));

// Count tasks for today (requests pending in assigned kebeles)
const [taskRows] = await pool.query(
  `SELECT COUNT(*) as count
  FROM customer_requests cr
  JOIN association_kebeles ak ON cr.kebele_id = ak.kebele_id
  WHERE ak.association_id = ?
  AND cr.status = 'Pending'
  AND DATE(cr.created_at) = CURDATE()`,
  [association.id]
);

```



```

);

const tasksToday = taskRows[0]?.count || 0;

res.json({
  success: true,
  data: {
    association_name: association.name,
    assigned_kebeles: assignedKebeles,
    tasks_today: tasksToday
  }
});
} catch (err) {
  console.error("GET ASSOCIATION PROFILE ERROR:", err);
  res.status(500).json({
    success: false,
    error: err.message || "Internal server error"
  });
}
};

// Send notification
const sendNotification = async (req, res) => {
  try {
    const userId = req.user.id;
    const { title, message, target_audience, kebele_id } = req.body;

    if (!title || !message) {
      return res.status(400).json({
        success: false,
        error: "Title and message are required"
      });
    }

    // Check if user is approved
    const [userRows] = await pool.query(
      "SELECT id, is_approved FROM users WHERE id = ?",
      [userId]
    );

    if (!userRows.length) {
      return res.status(404).json({
        success: false,
        error: "User not found"
      });
    }

    const user = userRows[0];

    // Check approval status
    if (!user.is_approved) {

```

```

    return res.status(403).json({
      success: false,
      error: "Your account is pending approval. Please wait for admin approval before sending
notifications.",
      pending_approval: true
    });
  }

  // Get association id
  const [assocRows] = await pool.query(
    "SELECT id FROM associations WHERE leader_id = ? LIMIT 1",
    [userId]
  );

  if (!assocRows.length) {
    return res.status(404).json({
      success: false,
      error: "Association not found for this user. Please contact administrator."
    });
  }

  const associationId = assocRows[0].id;

  // Insert notification
  const [result] = await pool.query(
    `INSERT INTO notifications (association_id, title, message, target_audience, kebele_id)
VALUES (?, ?, ?, ?, ?)`,
    [
      associationId,
      title,
      message,
      target_audience || "ALL",
      kebele_id || null
    ]
  );

  res.status(201).json({
    success: true,
    message: "Notification sent successfully",
    data: {
      notification_id: result.insertId
    }
  });
} catch (err) {
  console.error("SEND NOTIFICATION ERROR:", err);
  res.status(500).json({
    success: false,
    error: err.message || "Internal server error"
  });
}
};

```

```
module.exports = {  
  getProfile,  
  sendNotification,  
};
```

Chapter Five: Testing and Evaluation

5.1 Overview of Testing and Evaluation

Testing and evaluation are critical activities carried out to ensure that the **Digital Waste Management and Recycling System** operates according to its specified requirements and design objectives. These processes help verify that the system performs its intended functions correctly and consistently before it is deployed for real-world use.

Testing focuses on validating both **functional requirements** and **non-functional requirements**. Functional testing ensures that core system features—such as user registration, waste reporting, scheduling, route assignment, notifications, and reporting—work as expected. Non-functional testing evaluates quality attributes including **performance, security, reliability, usability, scalability, and maintainability**, which are essential for a system that serves multiple users and handles real-time data.

The primary goal of testing is to confirm that the system:

- **Works correctly:** All system functions produce accurate and expected results without errors.
- **Is secure:** User data and system operations are protected against unauthorized access, misuse, and common security threats.
- **Operates efficiently:** The system responds quickly to user requests and performs well even when many users are active simultaneously.
- **Is reliable:** The system remains stable, available, and consistent over time, with minimal failures or downtime under normal and peak usage conditions.

Evaluation complements testing by analyzing test results to determine whether the system meets predefined quality standards and project objectives. It helps identify weaknesses, measure system performance, and assess readiness for deployment. Through systematic testing and evaluation, potential defects are detected early, system quality is improved, and confidence is built in the system's ability to support efficient and demand-based waste management operations in urban environments.

The testing process focuses on:

- Correctness of individual system components
- Proper interaction between integrated modules
- System performance and scalability
- Security and access control
- Maintainability and reliability

The following testing types were applied:

- **Unit Testing**
- **Integration Testing**
- **System Testing**
- **Security Testing**

5.2 Testing Parameters (Quality Attributes)

To ensure that the Digital Waste Management and Recycling System delivers a high level of quality and reliability, testing was conducted based on several key quality attributes. These testing parameters evaluate not only whether the system functions correctly, but also how well it performs under real-world conditions and long-term usage.

Performance refers to how efficiently the system responds to user actions and processes requests. In this system, performance testing evaluates response time when users submit waste reports, generate schedules, or view dashboards. It also measures how quickly the system processes multiple requests simultaneously without delays. High performance is essential to ensure timely waste collection, real-time monitoring, and smooth interaction for citizens, waste collectors, and municipal staff.

Maintainability measures how easily the system can be modified, updated, or repaired when changes are required. This includes fixing software bugs, updating business rules, adding new features, or improving system performance. The Digital Waste Management System is designed using modular components and clear code structure, making it easier for developers to maintain and enhance the system over time without disrupting existing functionality.

Security ensures that the system protects sensitive information and prevents unauthorized access. Security testing focuses on user authentication, role-based access control, data encryption, and protection against common cyber threats such as SQL injection, unauthorized login attempts, and data manipulation. Given that the system stores personal user data and operational information, strong security mechanisms are critical to maintain trust and ensure safe system operation.

Reliability: refers to the system's ability to operate continuously and consistently without failure. Reliability testing evaluates whether the system remains stable during extended operation, handles unexpected errors gracefully, and maintains accurate data records. A reliable system ensures that waste collection schedules, reports, and monitoring data are always available and trustworthy, even during high usage or partial system failures.

Usability: focuses on how easy and intuitive the system is for its users, including citizens, waste collectors, and municipal staff. Usability testing examines whether users can complete tasks such as submitting waste reports, updating collection status, or generating reports with minimal effort and confusion. A user-friendly interface reduces training requirements, improves user satisfaction, and encourages active participation in the waste management process.

Scalability: measures the system's ability to handle growth in users, data volume, and operational complexity. Scalability testing evaluates how well the system performs when the number of citizens, waste requests, vehicles, or collection routes increases. This attribute is particularly important for urban environments, where population growth and changing waste patterns demand a system that can expand without significant performance degradation or redesign.

5.3 Unit Testing

Unit testing is performed to verify the correctness of individual components or classes of the Digital Waste Management and Recycling System in isolation. Each unit is tested independently to ensure that it performs its intended function accurately according to the system requirements. By testing components separately, errors can be detected and corrected early in the development process before they affect other parts of the system.

This type of testing focuses on the internal logic of modules such as data validation, calculations, decision rules, and method outputs. Unit testing improves system quality by ensuring reliability, reducing debugging time during later stages, and supporting maintainability through well-tested and modular code.

The following system units were tested:

The **User Authentication Module** was tested to verify correct login, logout, and role-based access control. Tests ensured that valid users can access the system while invalid credentials are rejected, helping to maintain system security.

The **Waste Request Module** was tested to confirm that citizens can submit waste collection requests correctly with valid location, waste type, and quantity information. This testing ensured that requests are stored accurately and error handling works properly for invalid inputs.

The **Scheduling Module** was tested to verify the generation of dynamic collection schedules based on reported demand and available resources. Unit tests ensured that schedules are created correctly and updated when new requests are added.

The **Notification Module** was tested to ensure that alerts and notifications are sent accurately to citizens, waste collectors, and municipal staff at the appropriate times. This helps maintain usability and effective communication within the system.

Unit Test Cases

Table 4: Unit Test Cases

Test Case ID	Unit	Test Description	Input	Expected Result	Parameter
UT-01	Login Module	Validate user login	Valid username & password	User logged in successfully	Security
UT-02	Login Module	Invalid login attempt	Wrong password	Access denied	Security

UT-03	Waste Request	Submit waste report	Location & waste type	Request saved	Reliability
UT-04	Schedule Generator	Generate schedule	Valid requests	Schedule created	Performance
UT-05	Notification	Send alert	Completed collection	Notification sent	Usability

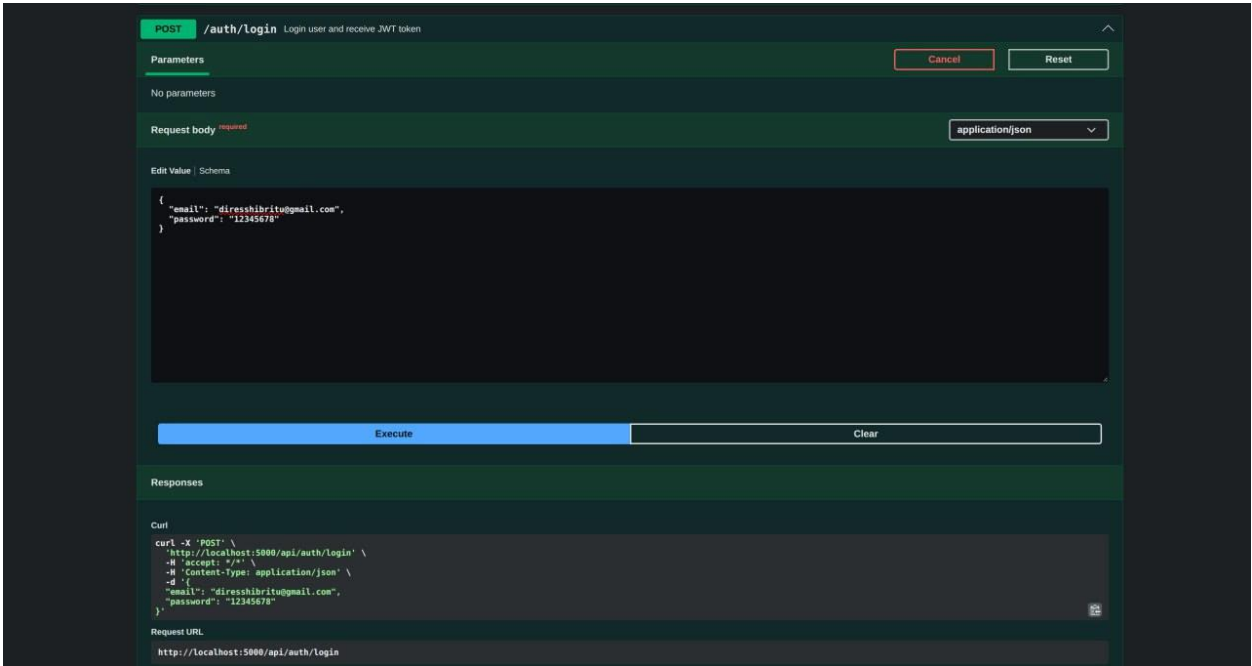


Figure 16:UT-01

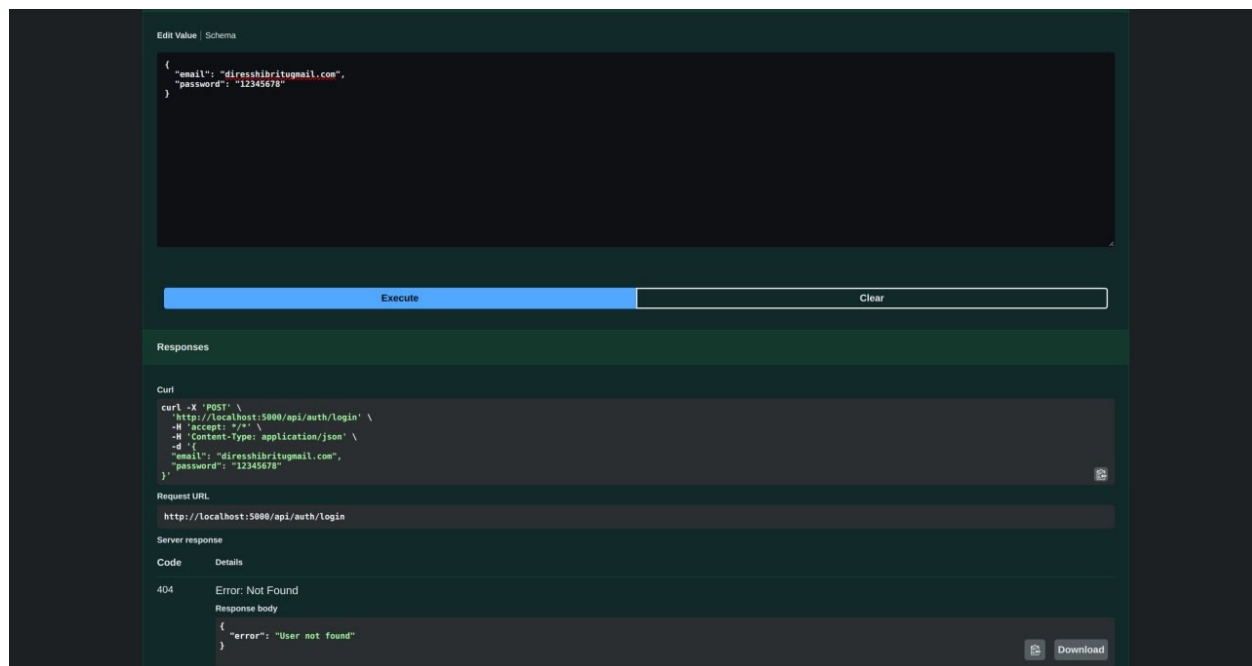


Figure 17:U-02

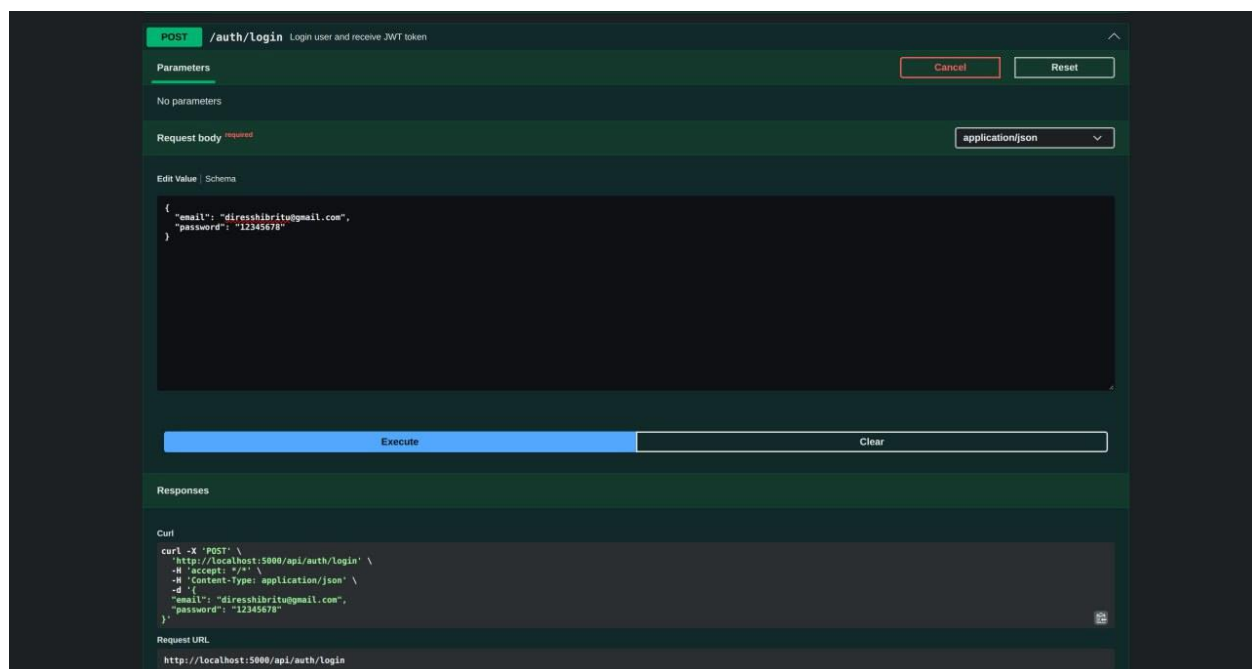


Figure 18:UT-03

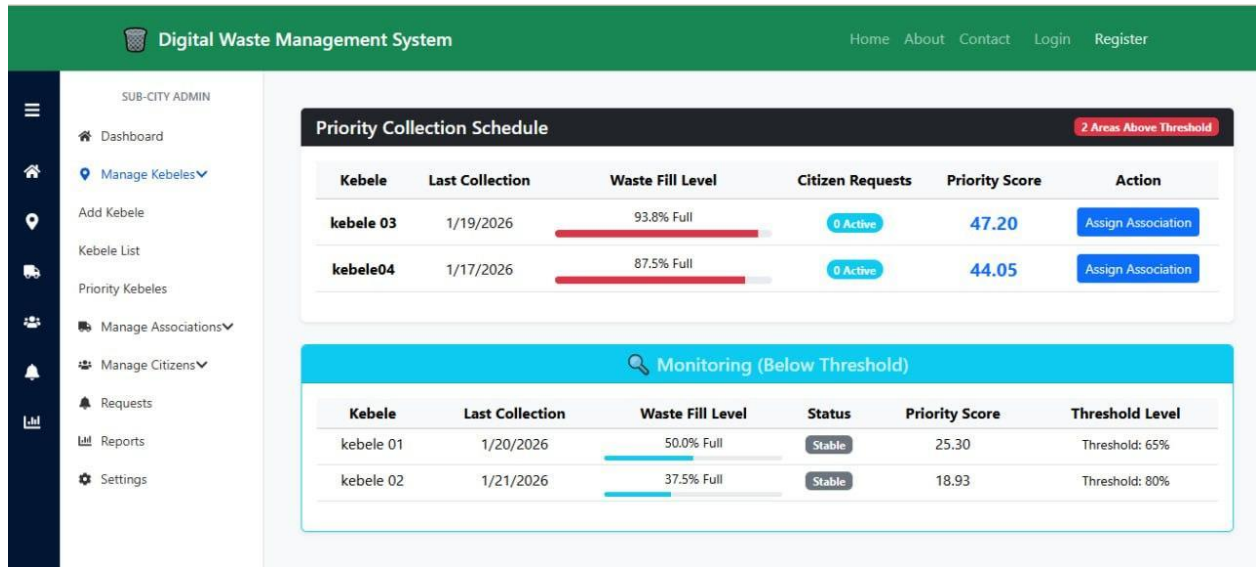


Figure 19:UT-04

5.4 Integration Testing

Integration testing is carried out to verify that different modules of the Digital Waste Management and Recycling System work together correctly after they have been individually tested through unit testing. The main objective of this testing is to ensure that data and control flow smoothly between integrated modules and that system operations function correctly when components interact as part of a complete workflow.

This type of testing focuses on detecting issues related to incorrect data exchange, interface mismatches, and communication failures between modules. Since the system relies on real-time coordination between citizens, waste collectors, and municipal staff, proper integration is essential for maintaining system reliability and operational efficiency.

The following module integrations were tested:

The integration between the **Waste Request Module and Scheduling Module** was tested to ensure that waste collection requests submitted by citizens are correctly transferred to the scheduling component. This verifies that new requests are automatically considered when generating demand-based collection schedules and that priority rules are applied correctly.

The interaction between the **Scheduling Module and Collector Assignment Module** was tested to confirm that generated schedules are accurately assigned to available waste collectors and vehicles. This ensures that collectors receive correct route information and task details without delay or data inconsistency.

The integration between the **Collection Status Module and Notification Module** was tested to ensure that status updates made by waste collectors (such as "Collected" or "Completed") trigger timely notifications to citizens and municipal staff. This improves transparency and keeps all stakeholders informed about service progress.

The integration between the **Reports Module and Database Module** was tested to verify that

stored data related to waste requests, collection activities, and recycling operations is correctly retrieved and processed to generate accurate performance and analytical reports.

Overall, integration testing confirmed that the interconnected system modules operate harmoniously as a single system, ensuring data consistency, reliable communication, and smooth execution of the digital waste management workflow.

Integration Test Cases

Table 5:Sample Integration Test CasesTable 6:System Test Case

Test Case ID	Modules Involved	Test Description	Expected Result	Parameter
IT-01	User + Waste Request	Submit request	Request stored correctly	Reliability
IT-02	Request+ Scheduling	Auto scheduling	Request assigned	Performance
IT-03	Schedule + Collector	Route assignment	Collector notified	Usability
IT-04	Collection + Notification	Status update	Citizen notified	Reliability
IT-05	Database + Report	Generate report	Accurate report	Maintainability

5.5 System Testing

System testing is conducted to evaluate the Digital Waste Management and Recycling System as a complete and fully integrated solution. Unlike unit and integration testing, which focus on individual components and their interactions, system testing verifies that the entire system functions correctly as a whole in a real-world operating environment. The main objective is to ensure that all functional and non-functional requirements are satisfied according to the project specifications.

This testing phase simulates actual usage scenarios involving citizens, waste collectors, municipal staff, and administrators. It validates whether the system supports demand-based waste management operations effectively, from waste reporting to collection, monitoring, and reporting. System testing also ensures that the system behaves correctly under normal and peak usage conditions.

The following key focus areas were evaluated:

End-to-end workflows were tested to verify that complete processes—from a citizen submitting a waste collection request to the waste being collected and reported—operate smoothly without errors. This ensures continuity of operations and confirms that all system modules work together as intended.

Performance under load was tested to evaluate how the system behaves when multiple users access it simultaneously. This includes testing response times, scheduling efficiency, and system stability during high volumes of waste requests, ensuring that the system remains responsive and efficient during peak usage periods.

Role-based access was tested to ensure that different user roles (citizens, waste collectors, municipal staff, and administrators) can only access authorized system features and data. This confirms that access control rules are enforced correctly and system security policies are upheld.

User experience was tested to evaluate the usability and clarity of the system interfaces. This includes assessing how easily users can navigate the system, complete tasks, and understand system feedback such as notifications and status updates. A positive user experience is essential to encourage active participation and effective system adoption.

- ❖ Overall, system testing confirmed that the Digital Waste Management and Recycling System meets its design objectives, performs reliably in realistic scenarios, and is ready for operational deployment.

System Test Cases

Table 7: System Test Case

Test Case ID	Scenario	Expected Result	Parameter
ST-01	Citizen submits request	Request scheduled	Usability
ST-02	Collector completes route	Status updated	Reliability
ST-03	Admin monitors system	Live data shown	Performance
ST-04	Multiple users active	No system crash	Scalability
ST-05	Report generation	Correct analytics	Maintainability

5.6 Security Testing

Security testing is conducted to ensure that the Digital Waste Management and Recycling System protects its data, resources, and operations from unauthorized access, malicious attacks, and misuse. Since the system handles sensitive user information and critical municipal operations, strong security measures are essential to maintain data integrity, user trust, and system reliability.

This testing phase focuses on verifying that security controls are correctly implemented and enforced throughout the system. It ensures that only authorized users can access specific system functionalities and that the system is resilient against common security threats.

The following security focus areas were evaluated:

Authentication and authorization were tested to ensure that users must provide valid credentials to access the system and that role-based access control is enforced correctly. Tests verified that citizens, waste collectors, municipal staff, and administrators can only access functions permitted by their roles, while unauthorized access attempts are blocked.

Data protection was tested to ensure that sensitive information such as user credentials, personal data, and operational records are securely stored and transmitted. This includes verifying encryption mechanisms, secure session management, and proper handling of system logs to prevent data leakage or unauthorized modification.

Input validation was tested to confirm that the system properly handles user input and prevents malicious data from being processed. This includes protection against common attacks such as SQL injection, cross-site scripting (XSS), and invalid data submission. Ensuring proper input validation helps maintain system stability and prevents security vulnerabilities.

- Overall, security testing confirmed that the system provides a secure operating environment, enforces strict access control, and safeguards data against threats, making it suitable for deployment in a real-world municipal setting.

Security Test Cases

Table 8:Security Test Cases

Test Case ID	Test Type	Description	Expected Result	Parameter
SEC-01	Authentication	Unauthorized login	Access denied	Security
SEC-02	Role Access	Citizen accessing admin page	Blocked	Security
SEC-03	Data Access	Encrypted storage	Data protected	Security

5.7 Evaluation Summary

The evaluation summary presents an overall assessment of the testing activities conducted for the Digital Waste Management and Recycling System. This evaluation is based on the results obtained from unit testing, integration testing, system testing, and security testing. The purpose of this summary is to confirm whether the system meets its functional and non-functional requirements and is ready for real-world deployment.

Unit testing was successfully completed, indicating that all individual system components performed their intended functions correctly when tested independently. Core modules such as authentication, waste request handling, scheduling, and notification operated as expected without critical defects.

Integration testing also passed successfully, demonstrating that different system modules interact and communicate effectively. Data flow between modules such as waste request and scheduling, scheduling and collector assignment, collection status and notification, and reports and database was consistent and error-free.

System testing confirmed that the complete system works correctly as a unified solution. End-to-end workflows functioned smoothly, performance remained stable under load, role-based access controls were properly enforced, and overall user experience met the expected standards.

Security testing passed successfully, verifying that the system is protected against unauthorized access, data misuse, and common security threats. Authentication, authorization, data protection, and input validation mechanisms were found to be effective.

- ❖ Overall, the evaluation results indicate that all testing phases were successfully completed, and the Digital Waste Management and Recycling System meets its design objectives and quality requirements. The system is therefore considered stable, secure, reliable, and suitable for deployment in a real-world municipal waste management environment.

Table 9: Test Result

Testing Type	Status
Unit Testing	✓Passed
Integration Testing	✓Passed
System Testing	✓Passed
Security Testing	✓Passed

REFERENCES

1. Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson.
2. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill Education.
3. Dennis, A., Wixom, B. H., & Roth, R. M. (2018). *Systems Analysis and Design* (6th Edition). Wiley.
4. Yourdon, E. (1989). *Modern Structured Analysis*. Prentice Hall.
5. Kendall, K. E., & Kendall, J. E. (2019). *Systems Analysis and Design* (10th Edition). Pearson.
6. PlantUML. (n.d.). *PlantUML Documentation*. Retrieved from <https://plantuml.com>
7. Figma. (n.d.). *Figma Prototyping Tool*. Retrieved from <https://www.figma.com>

Appendices

Questionnaire and Interview Topics

This appendix presents the questionnaire and interview topics used to gather requirements and understand the existing waste management practices, challenges, and stakeholder needs for the Digital Waste Management System.

1. Existing System: Municipal Waste Management

1. How is waste collection currently managed within the city and sub-cities?
2. What methods are used to receive waste collection requests from citizens?
3. How are waste collection schedules planned and communicated?
4. Are waste collection activities tracked or documented digitally? If yes, how?
5. What limitations exist in the current waste management system?

2. Administrative Structure: City and Sub-City Management

1. How are cities, sub-cities, and kebeles administratively organized?
2. What roles do sub-city administrations play in waste management operations?
3. How are decisions made regarding waste collection priorities?
4. What challenges do administrators face in monitoring waste collection activities?

3. Existing System: Waste Collection Associations

1. How many waste collection associations operate within each sub-city?
2. How are associations assigned to kebeles or waste collection tasks?
3. What resources (trucks, workers) are available to each association?
4. How do associations report task completion or operational issues?
5. What difficulties do associations face during waste collection operations?

4. Citizen Interaction and Service Requests

1. How do citizens currently request waste collection services?
2. How are citizen complaints or feedback handled?
3. Are citizens informed about the status of their requests?
4. What improvements do citizens expect from a digital waste management system?

5. Priority Handling and Waste Collection Challenges

1. What factors are considered when prioritizing waste collection in kebeles?
2. Are there kebeles with consistently high waste accumulation?
3. How does delayed waste collection impact public health and the environment?
4. What challenges affect timely and efficient waste collection?

6. Communication and Notification Mechanisms

1. How do administrators communicate with waste collection associations?
2. Are notifications currently used to inform associations about tasks?

3. What communication challenges exist between citizens, associations, and administrators?
4. How could digital notifications improve coordination and response time?

7. System Improvement Expectations

1. What features are most important in a digital waste management system?
2. How can the system improve transparency and accountability?
3. What reports or analytics are required for decision-making?
4. How can the system support future scalability and growth?